

Question 1: List all products

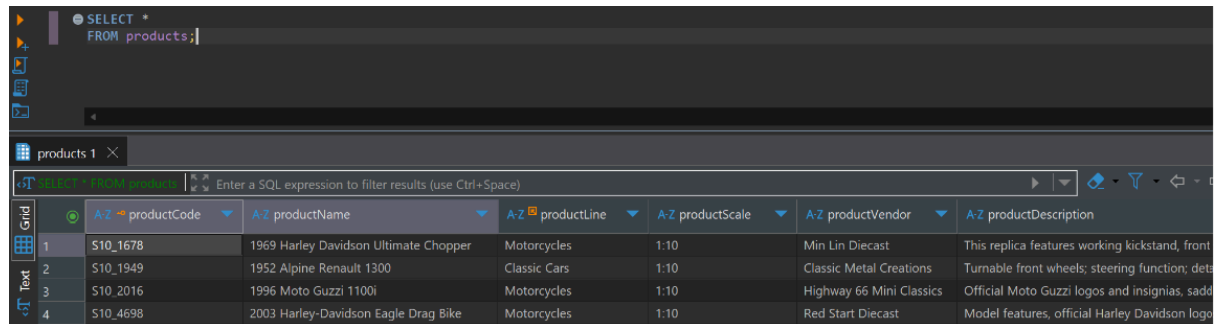
SQL Query

```
SELECT *  
FROM products;
```

Explanation

This retrieves all rows and all columns from the products table.

Result



The screenshot shows a SQL IDE interface. At the top, the query editor contains the text: `SELECT *
FROM products;`. Below the editor, a tab labeled 'products 1' is active. The results are displayed in a table with 7 columns: productCode, productName, productLine, productScale, productVendor, and productDescription. The table contains 4 rows of data.

	productCode	productName	productLine	productScale	productVendor	productDescription
1	S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	1:10	Min Lin Diecast	This replica features working kickstand, front
2	S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; steering function; deta
3	S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classics	Official Moto Guzzi logos and insignias, sadd
4	S10_4698	2003 Harley-Davidson Eagle Drag Bike	Motorcycles	1:10	Red Start Diecast	Model features, official Harley Davidson logo

JSON

```
{  
  "question": "List all products",  
  "sql": "SELECT * FROM products",  
  "intent": "retrieve all product records",  
  "tables": ["products"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 2: Get all customers

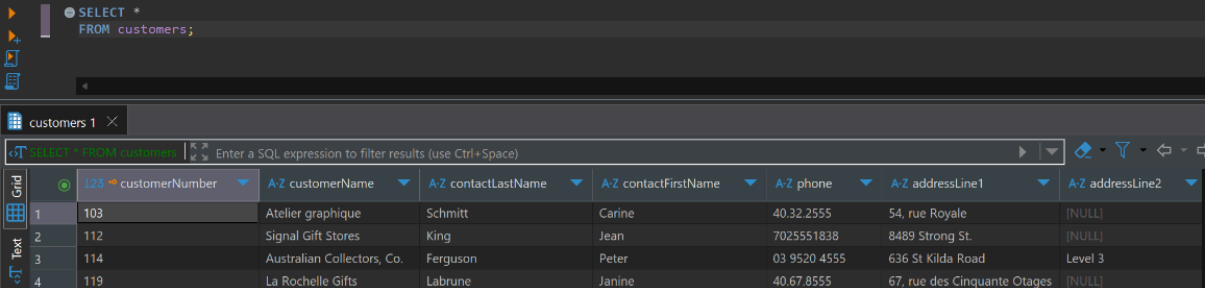
SQL Query

```
SELECT *  
FROM customers;
```

Explanation

This retrieves all rows and all columns from the customers table.

Result



The screenshot shows a database query interface. At the top, the SQL query `SELECT * FROM customers;` is entered. Below the query, a table titled "customers 1" displays the results. The table has 8 columns: `customerNumber`, `customerName`, `contactLastName`, `contactFirstName`, `phone`, `addressLine1`, and `addressLine2`. The results are as follows:

	customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1	addressLine2
1	103	Atelier graphique	Schmitt	Carine	40.32.2555	54, rue Royale	[NULL]
2	112	Signal Gift Stores	King	Jean	7025551838	8489 Strong St.	[NULL]
3	114	Australian Collectors, Co.	Ferguson	Peter	03 9520 4555	636 St Kilda Road	Level 3
4	119	La Rochelle Gifts	Labrun	Janine	40.67.8555	67, rue des Cinquante Otages	[NULL]

JSON

```
{  
  "question": "Get all customers",  
  "sql": "SELECT * FROM customers",  
  "intent": "retrieve all customer records",  
  "tables": ["customers"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 3: Show all orders

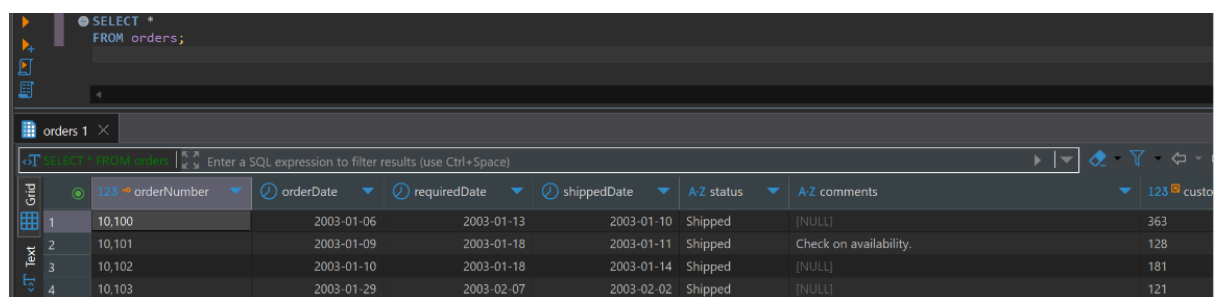
SQL Query

```
SELECT *  
FROM orders;
```

Explanation

This retrieves all rows and all columns from the orders table.

Result



The screenshot shows a database query tool interface. At the top, the SQL query `SELECT * FROM orders;` is entered. Below the query, a table view displays the results. The table has columns: `orderNumber`, `orderDate`, `requiredDate`, `shippedDate`, `status`, `comments`, and `customerID`. The results show four rows of order data.

	orderNumber	orderDate	requiredDate	shippedDate	status	comments	customerID
1	10,100	2003-01-06	2003-01-13	2003-01-10	Shipped	[NULL]	363
2	10,101	2003-01-09	2003-01-18	2003-01-11	Shipped	Check on availability.	128
3	10,102	2003-01-10	2003-01-18	2003-01-14	Shipped	[NULL]	181
4	10,103	2003-01-29	2003-02-07	2003-02-02	Shipped	[NULL]	121

JSON

```
{  
  "question": "Show all orders",  
  "sql": "SELECT * FROM orders",  
  "intent": "retrieve all order records",  
  "tables": ["orders"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 4: List all employees

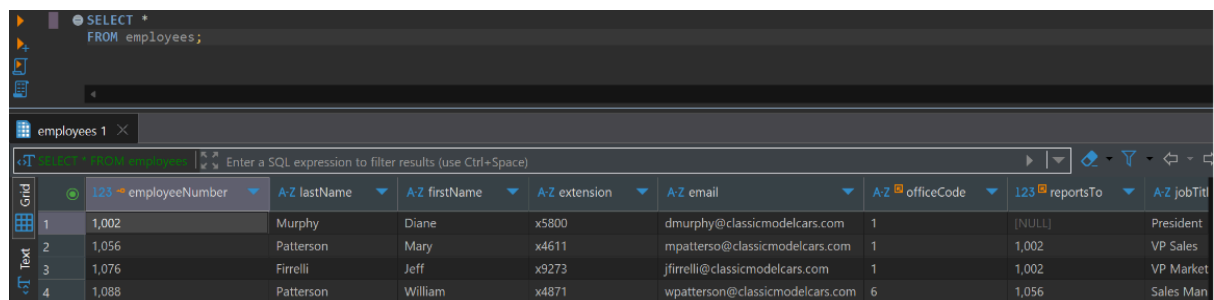
SQL Query

```
SELECT *  
FROM employees;
```

Explanation

This retrieves all rows and all columns from the employees table.

Result



The screenshot shows a database query tool interface. At the top, the SQL query `SELECT * FROM employees;` is entered. Below the query, a tab labeled 'employees 1' is active. A filter bar is present with the text 'Enter a SQL expression to filter results (use Ctrl+Space)'. The results are displayed in a table grid with the following columns: employeeNumber, lastName, firstName, extension, email, officeCode, reportsTo, and jobTitle. The data is as follows:

	employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1	1,002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	[NULL]	President
2	1,056	Patterson	Mary	x4611	mpatterson@classicmodelcars.com	1	1,002	VP Sales
3	1,076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1,002	VP Market
4	1,088	Patterson	William	x4871	wpatterson@classicmodelcars.com	6	1,056	Sales Man

JSON

```
{  
  "question": "List all employees",  
  "sql": "SELECT * FROM employees",  
  "intent": "retrieve all employee records",  
  "tables": ["employees"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 5: Get all offices

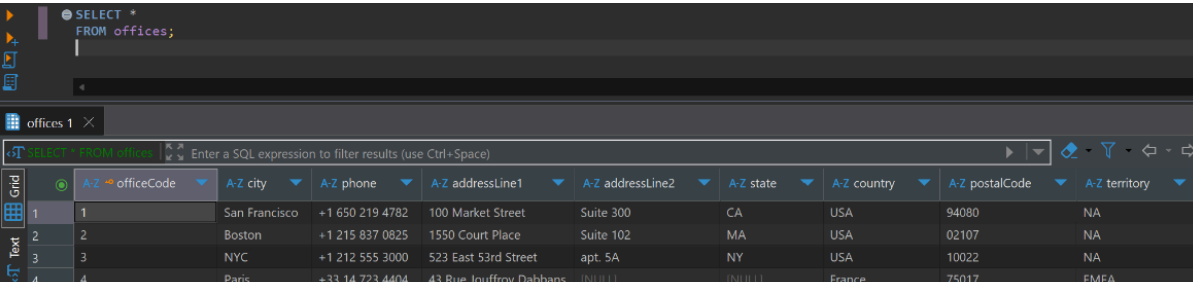
SQL Query

```
SELECT *  
FROM offices;
```

Explanation

This retrieves all rows and all columns from the offices table.

Result



The screenshot shows a database interface with a query editor at the top containing the SQL query `SELECT * FROM offices;`. Below the editor, a table titled 'offices 1' displays the results. The table has 11 columns: officeCode, city, phone, addressLine1, addressLine2, state, country, postalCode, and territory. There are 4 rows of data representing offices in San Francisco, Boston, NYC, and Paris.

	officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
1	1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
2	2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
3	3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA
4	4	Paris	+33 14 723 4404	43 Rue Jouffroy Dabbans	[NULL]	[NULL]	France	75017	EMEA

JSON

```
{  
  "question": "Get all offices",  
  "sql": "SELECT * FROM offices",  
  "intent": "retrieve all office records",  
  "tables": ["offices"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 6: Show all product lines

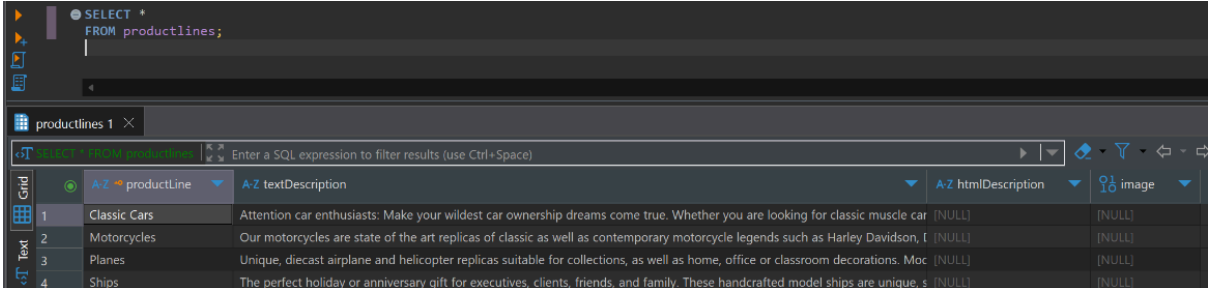
SQL Query

```
SELECT *  
FROM productlines;
```

Explanation

This retrieves all rows and all columns from the productlines table.

Result



The screenshot shows a database client interface. At the top, the SQL query `SELECT * FROM productlines;` is entered. Below the query editor, the results are displayed in a table view. The table has four columns: `productLine`, `textDescription`, `htmlDescription`, and `image`. The results are as follows:

	productLine	textDescription	htmlDescription	image
1	Classic Cars	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic muscle car	[NULL]	[NULL]
2	Motorcycles	Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, E	[NULL]	[NULL]
3	Planes	Unique, diecast airplane and helicopter replicas suitable for collections, as well as home, office or classroom decorations. Moc	[NULL]	[NULL]
4	Ships	The perfect holiday or anniversary gift for executives, clients, friends, and family. These handcrafted model ships are unique, s	[NULL]	[NULL]

JSON

```
{  
  "question": "Show all product lines",  
  "sql": "SELECT * FROM productlines",  
  "intent": "retrieve all product line records",  
  "tables": ["productlines"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 7: List all payments

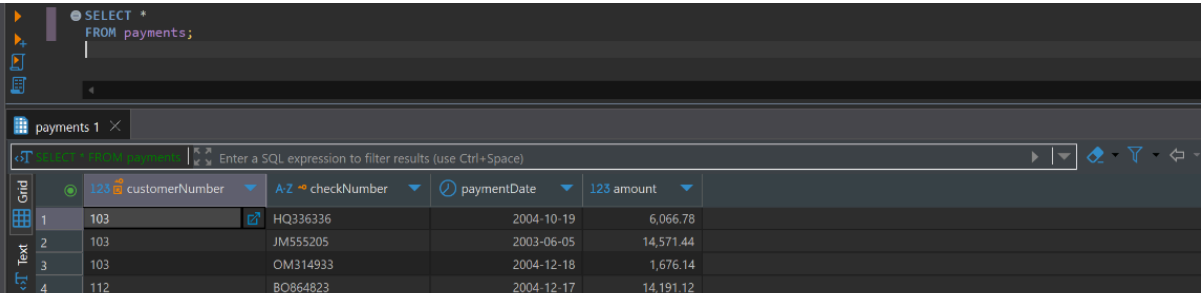
SQL Query

```
SELECT *  
FROM payments;
```

Explanation

This retrieves all rows and all columns from the payments table.

Result



The screenshot shows a database query tool interface. At the top, the SQL query `SELECT * FROM payments;` is entered. Below the query editor, a tab labeled 'payments 1' is active. The results are displayed in a table with 5 columns: 'customerNumber', 'checkNumber', 'paymentDate', and 'amount'. The table contains 4 rows of data. The interface also shows a filter bar with the text 'Enter a SQL expression to filter results (use Ctrl+Space)'.

	customerNumber	checkNumber	paymentDate	amount
1	103	HQ336336	2004-10-19	6,066.78
2	103	JM555205	2003-06-05	14,571.44
3	103	OM314933	2004-12-18	1,676.14
4	112	BO864823	2004-12-17	14,191.12

JSON

```
{  
  "question": "List all payments",  
  "sql": "SELECT * FROM payments",  
  "intent": "retrieve all payment records",  
  "tables": ["payments"],  
  "columns": ["*"],  
  "filters": [],  
  "joins": []  
}
```

Question 8: Get product names and prices

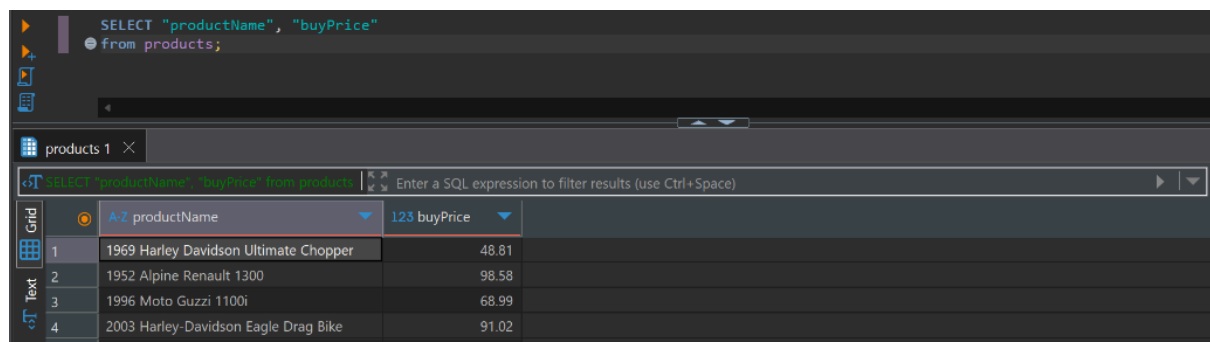
SQL Query

```
SELECT "productName", "buyPrice"
from products;
```

Explanation

This retrieves product names and their buy prices from the products table.

Result



The screenshot shows a database IDE with a SQL query editor at the top and a results pane below. The query is: `SELECT "productName", "buyPrice" from products;`. The results pane shows a table with 4 rows and 2 columns: `productName` and `buyPrice`. The data is as follows:

	productName	buyPrice
1	1969 Harley Davidson Ultimate Chopper	48.81
2	1952 Alpine Renault 1300	98.58
3	1996 Moto Guzzi 1100i	68.99
4	2003 Harley-Davidson Eagle Drag Bike	91.02

JSON

```
{
  "question": "Get product names and prices",
  "sql": "SELECT \"productName\", \"buyPrice\" FROM products",
  "intent": "retrieve product pricing information",
  "tables": ["products"],
  "columns": ["productName", "buyPrice"],
  "filters": [],
  "joins": []
}
```


Question 9: Get customer names and cities

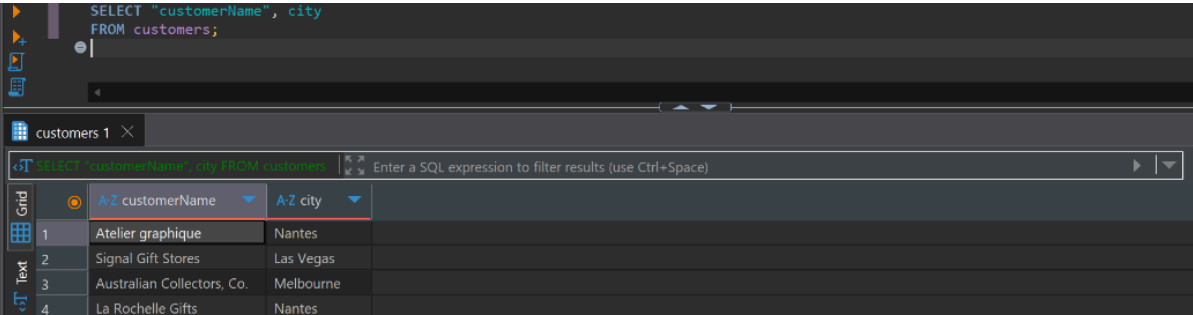
SQL Query

```
SELECT "customerName", city
FROM customers;
```

Explanation

This retrieves customer names and their corresponding cities.

Result



The screenshot shows a database IDE with a SQL query editor at the top containing the query: `SELECT "customerName", city FROM customers;`. Below the editor, a tab labeled "customers 1" is active. A search bar contains the query text. The results are displayed in a grid view with columns "A-Z customerName" and "A-Z city".

	A-Z customerName	A-Z city
1	Atelier graphique	Nantes
2	Signal Gift Stores	Las Vegas
3	Australian Collectors, Co.	Melbourne
4	La Rochelle Gifts	Nantes

JSON

```
{
  "question": "Get customer names and cities",
  "sql": "SELECT \"customerName\", city FROM customers",
  "intent": "retrieve customer location information",
  "tables": ["customers"],
  "columns": ["customerName", "city"],
  "filters": [],
  "joins": []
}
```

Question 10: List employee first and last names

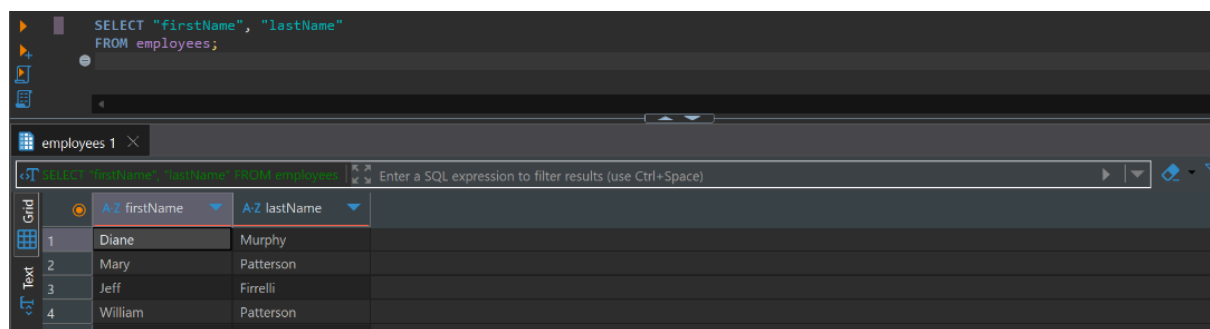
SQL Query

```
SELECT "firstName", "lastName"  
FROM employees;
```

Explanation

This retrieves first and last names of all employees.

Result



The screenshot shows a database query tool interface. At the top, the SQL query is displayed: `SELECT "firstName", "lastName" FROM employees;`. Below the query, the results are shown in a grid view. The grid has two columns: "A-Z firstName" and "A-Z lastName". The results are as follows:

	A-Z firstName	A-Z lastName
1	Diane	Murphy
2	Mary	Patterson
3	Jeff	Firrelli
4	William	Patterson

JSON

```
{  
  "question": "List employee first and last names",  
  "sql": "SELECT \"firstName\", \"lastName\" FROM employees",  
  "intent": "retrieve employee names",  
  "tables": ["employees"],  
  "columns": ["firstName", "lastName"],  
  "filters": [],  
  "joins": []  
}
```

Question 11: Get all order dates

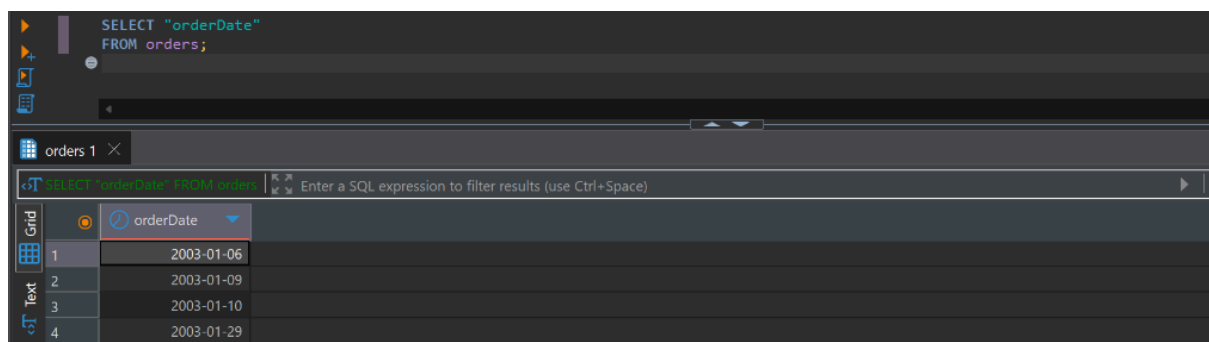
SQL Query

```
SELECT "orderDate"  
FROM orders;
```

Explanation

This retrieves all order dates from the orders table.

Result



The screenshot shows a database interface with a SQL editor at the top containing the query: `SELECT "orderDate" FROM orders;`. Below the editor, a tab labeled "orders 1" is active. The results are displayed in a table grid with the following data:

	orderDate
1	2003-01-06
2	2003-01-09
3	2003-01-10
4	2003-01-29

JSON

```
{  
  "question": "Get all order dates",  
  "sql": "SELECT \"orderDate\" FROM orders",  
  "intent": "retrieve all order dates",  
  "tables": ["orders"],  
  "columns": ["orderDate"],  
  "filters": [],  
  "joins": []  
}
```

Question 12: Show product vendor list

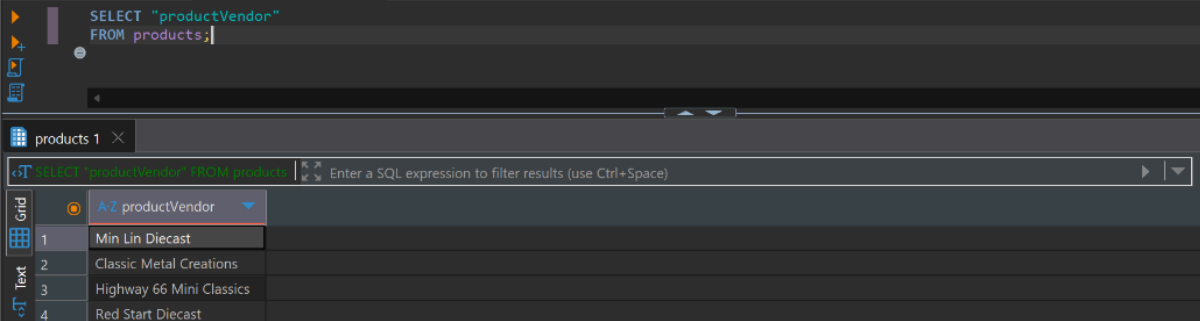
SQL Query

```
SELECT "productVendor"  
FROM products;
```

Explanation

This retrieves all product vendor names from the products table.

Result



The screenshot shows a database IDE interface. At the top, the SQL query is entered in a text editor: `SELECT "productVendor" FROM products;`. Below the editor, a tab labeled "products 1" is active. A search bar contains the query text. The results are displayed in a grid view with the following data:

	A-Z productVendor
1	Min Lin Diecast
2	Classic Metal Creations
3	Highway 66 Mini Classics
4	Red Start Diecast

JSON

```
{  
  "question": "Show product vendor list",  
  "sql": "SELECT \"productVendor\" FROM products",  
  "intent": "retrieve product vendors",  
  "tables": ["products"],  
  "columns": ["productVendor"],  
  "filters": [],  
  "joins": []  
}
```

Question 13: Get all product codes

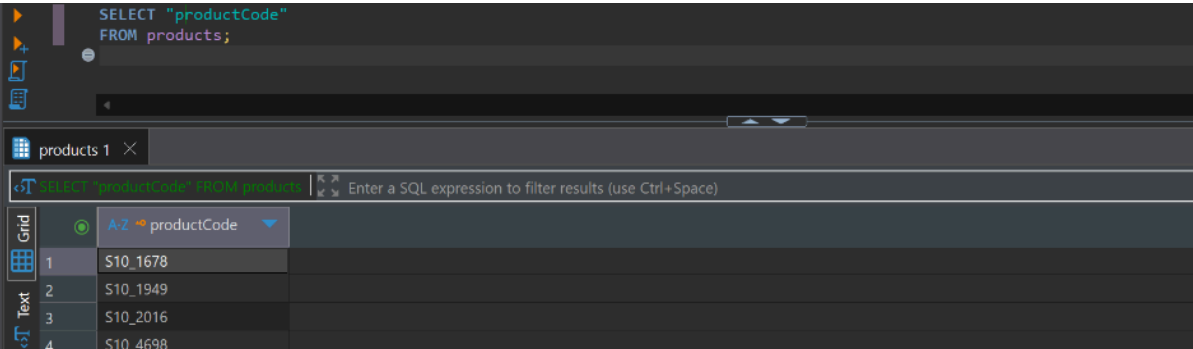
SQL Query

```
SELECT "productCode"
FROM products;
```

Explanation

This retrieves all product codes from the products table.

Result



The screenshot shows a database query tool interface. At the top, the SQL query is entered: `SELECT "productCode" FROM products;`. Below the query editor, a tab labeled "products 1" is active. A filter bar shows the same query. The results are displayed in a table grid with the following data:

	productCode
1	S10_1678
2	S10_1949
3	S10_2016
4	S10_4698

JSON

```
{
  "question": "Get all product codes",
  "sql": "SELECT \"productCode\" FROM products",
  "intent": "retrieve product identifiers",
  "tables": ["products"],
  "columns": ["productCode"],
  "filters": [],
  "joins": []
}
```

Question 14: List all countries from offices

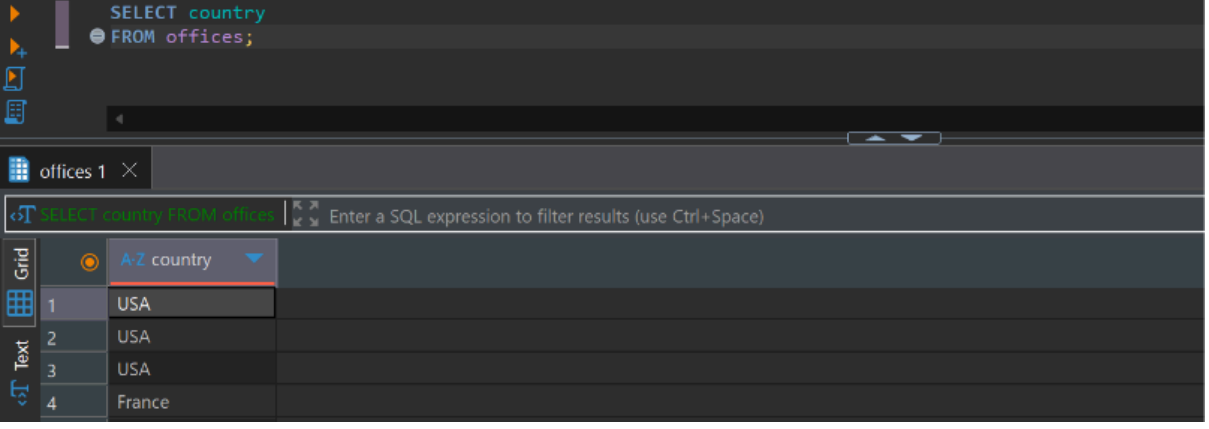
SQL Query

```
SELECT country
FROM offices;
```

Explanation

This retrieves all countries from the offices table.

Result



The screenshot shows a SQL IDE interface. At the top, the query editor contains the SQL query: `SELECT country FROM offices;`. Below the query editor, a tab labeled 'offices 1' is active. The results are displayed in a table with 4 rows. The first three rows show 'USA' and the fourth row shows 'France'. The table has a 'Grid' view selected, and a dropdown menu is open showing 'A-Z country'.

	country
1	USA
2	USA
3	USA
4	France

JSON

```
{
  "question": "List all countries from offices",
  "sql": "SELECT \"country\" FROM offices",
  "intent": "retrieve office locations by country",
  "tables": ["offices"],
  "columns": ["country"],
  "filters": [],
  "joins": []
}
```

Question 15: Show all order statuses

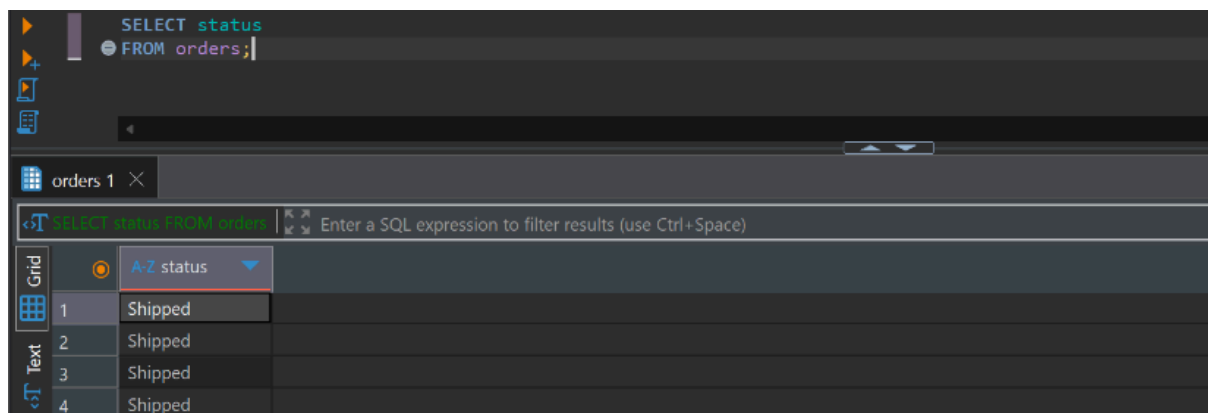
SQL Query

```
SELECT status  
FROM orders;
```

Explanation

This retrieves all order statuses from the orders table.

Result



The screenshot shows a database interface with a query editor at the top containing the SQL query `SELECT status FROM orders;`. Below the editor, a tab labeled 'orders 1' is active. The results are displayed in a table with a 'Grid' view selected. The table has 4 rows, all showing 'Shipped' status. A filter bar above the table shows 'A-Z status' and a search prompt 'Enter a SQL expression to filter results (use Ctrl+Space)'.

	A-Z status
1	Shipped
2	Shipped
3	Shipped
4	Shipped

JSON

```
{  
  "question": "Show all order statuses",  
  "sql": "SELECT \"status\" FROM orders",  
  "intent": "retrieve order status values",  
  "tables": ["orders"],  
  "columns": ["status"],  
  "filters": [],  
  "joins": []  
}
```

Question 16: Get all payment amounts

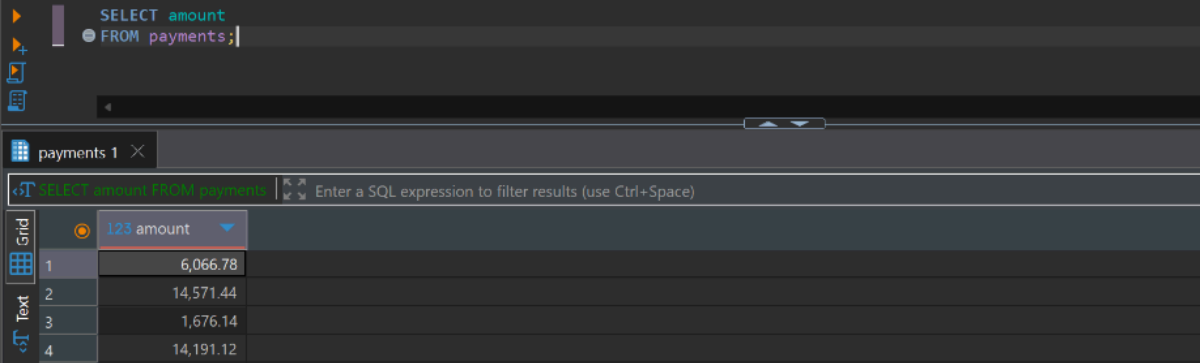
SQL Query

```
SELECT amount
FROM payments;
```

Explanation

This retrieves all payment amounts from the payments table.

Result



The screenshot shows a SQL query editor with the query `SELECT amount FROM payments;` entered. Below the query editor, a table titled "payments 1" is displayed. The table has a single column labeled "amount" and four rows of data. The first row has the value 6,066.78, the second 14,571.44, the third 1,676.14, and the fourth 14,191.12. The table is currently in "Grid" view.

	amount
1	6,066.78
2	14,571.44
3	1,676.14
4	14,191.12

JSON

```
{
  "question": "Get all payment amounts",
  "sql": "SELECT \"amount\" FROM payments",
  "intent": "retrieve payment amounts",
  "tables": ["payments"],
  "columns": ["amount"],
  "filters": [],
  "joins": []
}
```


Question 17: List all job titles

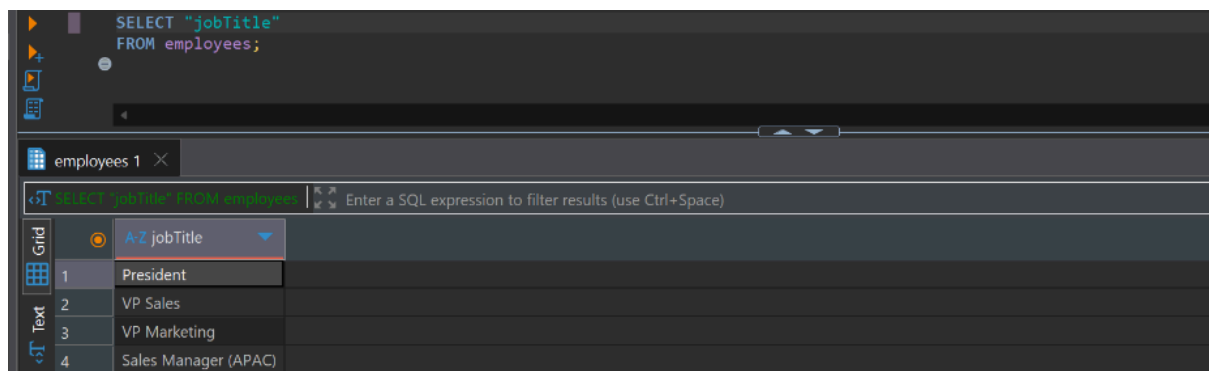
SQL Query

```
SELECT "jobTitle"  
FROM employees;
```

Explanation

This retrieves all job titles from the employees table.

Result



The screenshot shows a database query tool interface. At the top, the SQL query is displayed: `SELECT "jobTitle" FROM employees;`. Below the query, the results are shown in a table. The table has a single column labeled "jobTitle" and four rows of data. The first row is "President", the second is "VP Sales", the third is "VP Marketing", and the fourth is "Sales Manager (APAC)".

Grid	A-Z jobTitle
1	President
2	VP Sales
3	VP Marketing
4	Sales Manager (APAC)

JSON

```
{  
  "question": "List all job titles",  
  "sql": "SELECT \"jobTitle\" FROM employees",  
  "intent": "retrieve employee job roles",  
  "tables": ["employees"],  
  "columns": ["jobTitle"],  
  "filters": [],  
  "joins": []  
}
```

Question 18: Get customer phone numbers

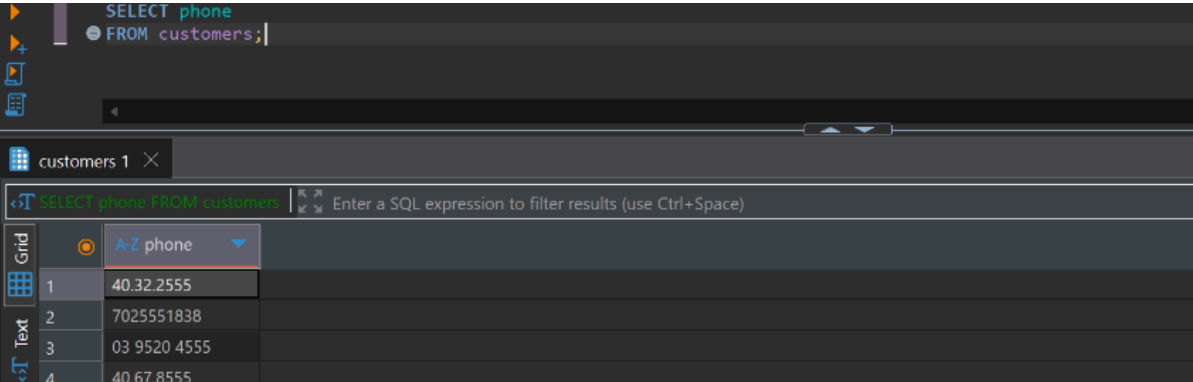
SQL Query

```
SELECT phone
FROM customers;
```

Explanation

This retrieves all customer phone numbers from the customers table.

Result



The screenshot shows a database interface with a query editor at the top containing the SQL query `SELECT phone FROM customers;`. Below the editor, a tab labeled 'customers 1' is active. The results are displayed in a table with a 'Grid' view selected. The table has two columns: an index and 'phone'. The data is as follows:

	phone
1	40.32.2555
2	7025551838
3	03 9520 4555
4	40.67.8555

JSON

```
{
  "question": "Get customer phone numbers",
  "sql": "SELECT \"phone\" FROM customers",
  "intent": "retrieve customer contact numbers",
  "tables": ["customers"],
  "columns": ["phone"],
  "filters": [],
  "joins": []
}
```

Question 19: Show product MSRP values

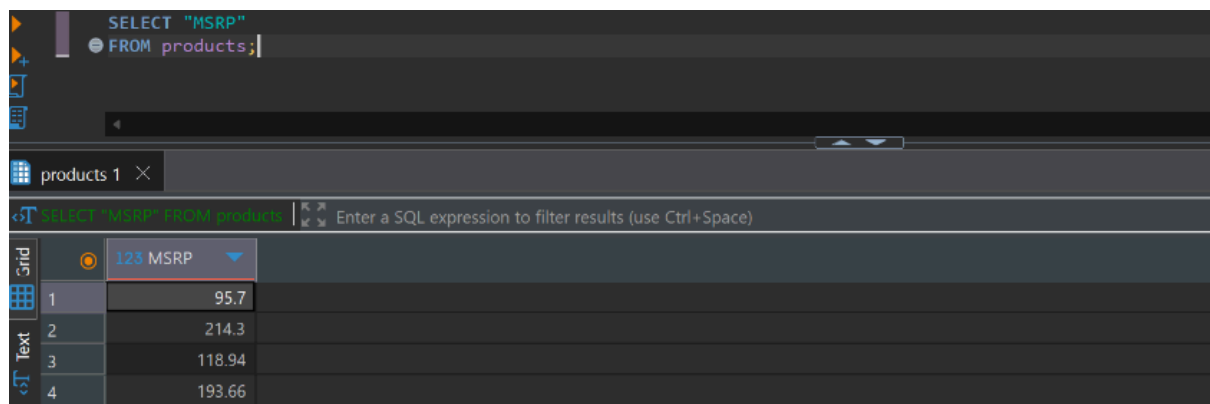
SQL Query

```
SELECT "MSRP"  
FROM products;
```

Explanation

This retrieves all MSRP values from the products table.

Result



The screenshot shows a SQL query editor with the query `SELECT "MSRP" FROM products;` entered. Below the query, a table titled "products 1" displays the results. The table has 4 rows and 1 column, with the column header "MSRP" and a value of 123. The rows contain the following MSRP values: 95.7, 214.3, 118.94, and 193.66.

	MSRP
1	95.7
2	214.3
3	118.94
4	193.66

JSON

```
{  
  "question": "Show product MSRP values",  
  "sql": "SELECT \"MSRP\" FROM products",  
  "intent": "retrieve product retail prices",  
  "tables": ["products"],  
  "columns": ["MSRP"],  
  "filters": [],  
  "joins": []  
}
```

Question 20: List order numbers

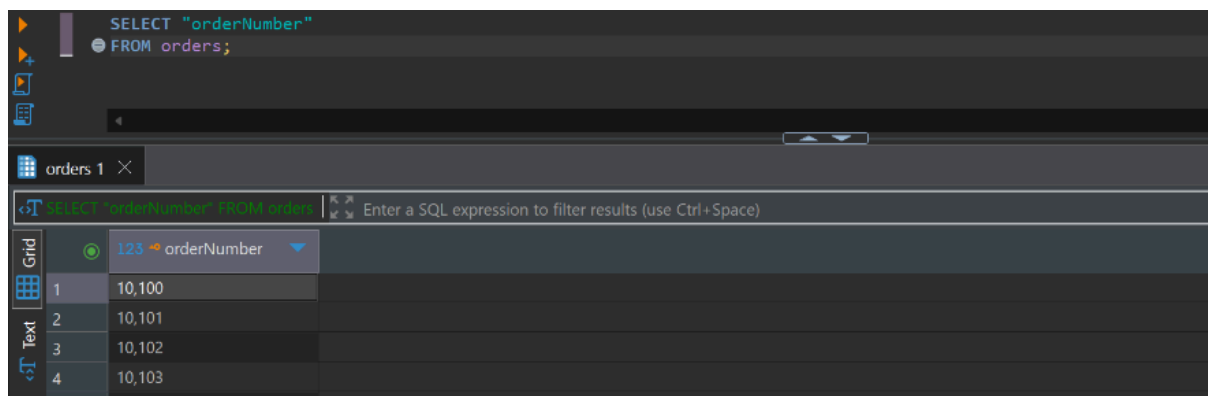
SQL Query

```
SELECT "orderNumber"  
FROM orders;
```

Explanation

This retrieves all order numbers from the orders table.

Result



The screenshot shows a database interface with a query editor at the top containing the SQL query: `SELECT "orderNumber" FROM orders;`. Below the editor, a tab labeled "orders 1" is active. The results are displayed in a table with a "Grid" view selected. The table has one column, "orderNumber", and four rows of data. The first row is highlighted.

	orderNumber
1	10,100
2	10,101
3	10,102
4	10,103

JSON

```
{  
  "question": "List order numbers",  
  "sql": "SELECT \"orderNumber\" FROM orders",  
  "intent": "retrieve order identifiers",  
  "tables": ["orders"],  
  "columns": ["orderNumber"],  
  "filters": [],  
  "joins": []  
}
```

Question 21: Get orders with customer names

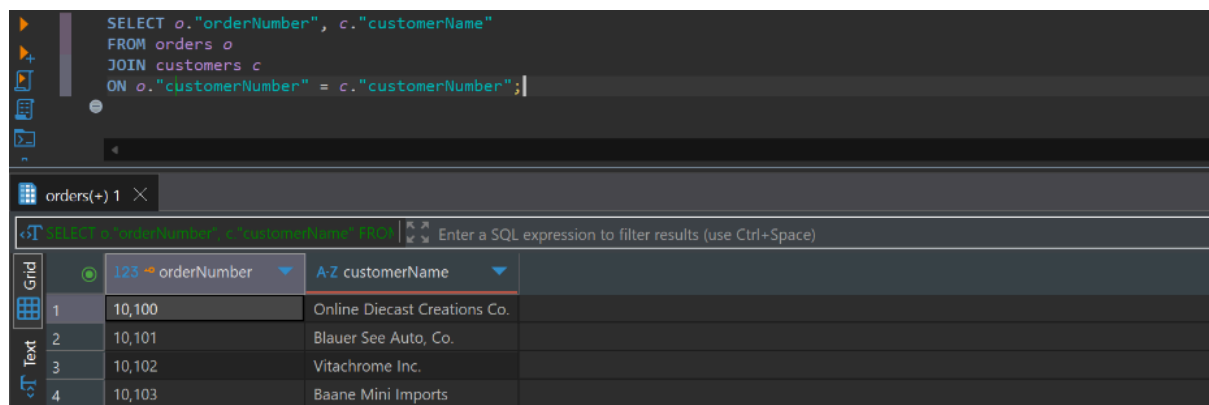
SQL Query

```
SELECT o."orderNumber", c."customerName"  
FROM orders o  
JOIN customers c  
ON o."customerNumber" = c."customerNumber";
```

Explanation

This retrieves each order along with the customer who placed it by joining orders and customers using customerNumber.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT o."orderNumber", c."customerName" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";`. The bottom pane shows the results in a table view. The table has two columns: `orderNumber` and `customerName`. The results are as follows:

	orderNumber	customerName
1	10,100	Online Diecast Creations Co.
2	10,101	Blauer See Auto, Co.
3	10,102	Vitachrome Inc.
4	10,103	Baane Mini Imports

JSON

```
{  
  "question": "Get orders with customer names",  
  "sql": "SELECT o.\"orderNumber\", c.\"customerName\" FROM orders o JOIN  
customers c ON o.\"customerNumber\" = c.\"customerNumber\"",  
  "intent": "retrieve orders along with customer information",  
  "tables": ["orders", "customers"],  
  "columns": ["orderNumber", "customerName"],  
  "filters": [],  
  "joins": ["orders.customerNumber = customers.customerNumber"]  
}
```

Question 22: Get employees with office city

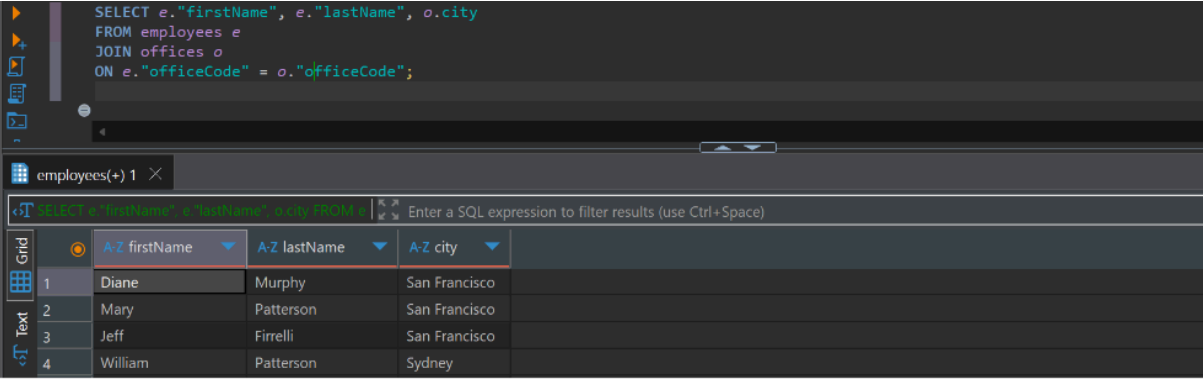
SQL Query

```
SELECT e."firstName", e."lastName", o.city
FROM employees e
JOIN offices o
ON e."officeCode" = o."officeCode";
```

Explanation

This retrieves employees along with their office city by joining employees and offices tables.

Result



The screenshot shows a SQL IDE interface. The top pane contains the SQL query: `SELECT e."firstName", e."lastName", o.city FROM employees e JOIN offices o ON e."officeCode" = o."officeCode";`. The bottom pane shows the results in a table grid. The table has 4 columns: `firstName`, `lastName`, and `city`. The results are as follows:

	firstName	lastName	city
1	Diane	Murphy	San Francisco
2	Mary	Patterson	San Francisco
3	Jeff	Firrelli	San Francisco
4	William	Patterson	Sydney

JSON

```
{
  "question": "Get employees with office city",
  "sql": "SELECT e.\"firstName\", e.\"lastName\", o.city FROM employees e JOIN\noffices o ON e.\"officeCode\" = o.\"officeCode\"",
  "intent": "retrieve employees with their office location",
  "tables": ["employees", "offices"],
  "columns": ["firstName", "lastName", "city"],
  "filters": [],
  "joins": ["employees.officeCode = offices.officeCode"]
}
```

Question 23: Get payments with customer names

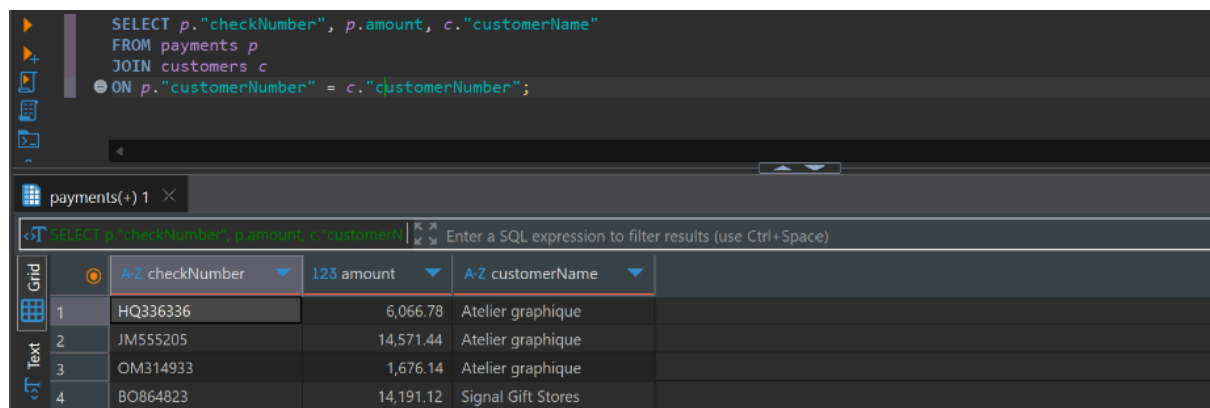
SQL Query

```
SELECT p."checkNumber", p.amount, c."customerName"  
FROM payments p  
JOIN customers c  
ON p."customerNumber" = c."customerNumber";
```

Explanation

This retrieves payments along with customer names using a join on customerNumber.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT p."checkNumber", p.amount, c."customerName" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";`. The bottom pane shows the results in a table view. The table has 4 rows and 3 columns: checkNumber, amount, and customerName. The results are as follows:

	A-Z checkNumber	123 amount	A-Z customerName
1	HQ336336	6,066.78	Atelier graphique
2	JM555205	14,571.44	Atelier graphique
3	OM314933	1,676.14	Atelier graphique
4	BO864823	14,191.12	Signal Gift Stores

JSON

```
{  
  "question": "Get payments with customer names",  
  "sql": "SELECT p.\"checkNumber\", p.\"amount\", c.\"customerName\" FROM  
payments p JOIN customers c ON p.\"customerNumber\" = c.\"customerNumber\"",  
  "intent": "retrieve payment details with customer information",  
  "tables": ["payments", "customers"],  
  "columns": ["checkNumber", "amount", "customerName"],  
  "filters": [],  
  "joins": ["payments.customerNumber = customers.customerNumber"]  
}
```

Question 24: Get order details with product names

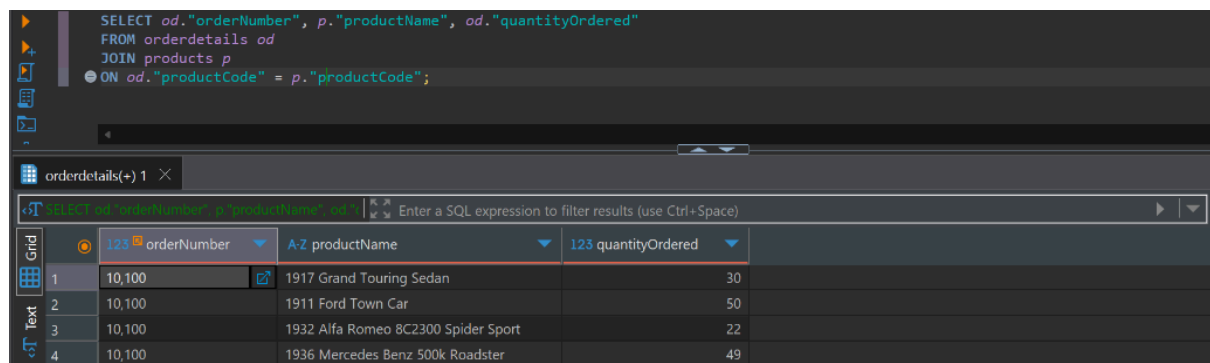
SQL Query

```
SELECT od."orderNumber", p."productName", od."quantityOrdered"
FROM orderdetails od
JOIN products p
ON od."productCode" = p."productCode";
```

Explanation

This retrieves order line items along with product names by joining orderdetails and products.

Result



The screenshot shows a database IDE with the following SQL query in the editor:

```
SELECT od."orderNumber", p."productName", od."quantityOrdered"
FROM orderdetails od
JOIN products p
ON od."productCode" = p."productCode";
```

Below the editor, a table grid displays the results of the query. The table has four columns: orderNumber, productName, and quantityOrdered. The results are as follows:

	orderNumber	productName	quantityOrdered
1	10,100	1917 Grand Touring Sedan	30
2	10,100	1911 Ford Town Car	50
3	10,100	1932 Alfa Romeo 8C2300 Spider Sport	22
4	10,100	1936 Mercedes Benz 500k Roadster	49

JSON

```
{
  "question": "Get order details with product names",
  "sql": "SELECT od.\"orderNumber\", p.\"productName\", od.\"quantityOrdered\"
FROM orderdetails od JOIN products p ON od.\"productCode\" = p.\"productCode\"",
  "intent": "retrieve order line items with product information",
  "tables": ["orderdetails", "products"],
  "columns": ["orderNumber", "productName", "quantityOrdered"],
  "filters": [],
  "joins": ["orderdetails.productCode = products.productCode"]
}
```


Question 25: Get products with product line description

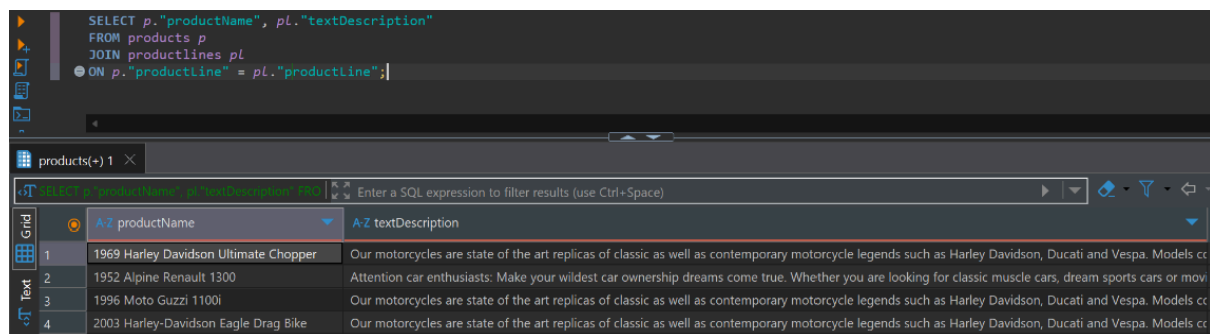
SQL Query

```
SELECT p."productName", pl."textDescription"
FROM products p
JOIN productlines pl
ON p."productLine" = pl."productLine";
```

Explanation

This retrieves products along with their product line descriptions by joining products and productlines.

Result



The screenshot shows a database IDE with the following SQL query in the editor:

```
SELECT p."productName", pl."textDescription"
FROM products p
JOIN productlines pl
ON p."productLine" = pl."productLine";
```

Below the editor, a table grid displays the results of the query. The table has two columns: 'AZ productName' and 'AZ textDescription'. The results are as follows:

	AZ productName	AZ textDescription
1	1969 Harley Davidson Ultimate Chopper	Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models cc
2	1952 Alpine Renault 1300	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic muscle cars, dream sports cars or movi
3	1996 Moto Guzzi 1100i	Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models cc
4	2003 Harley-Davidson Eagle Drag Bike	Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models cc

JSON

```
{
  "question": "Get products with product line description",
  "sql": "SELECT p.\"productName\", pl.\"textDescription\" FROM products p JOIN
productlines pl ON p.\"productLine\" = pl.\"productLine\"",
  "intent": "retrieve product details with product line information",
  "tables": ["products", "productlines"],
  "columns": ["productName", "textDescription"],
  "filters": [],
  "joins": ["products.productLine = productlines.productLine"]
}
```

Question 26: Get customers with sales rep names

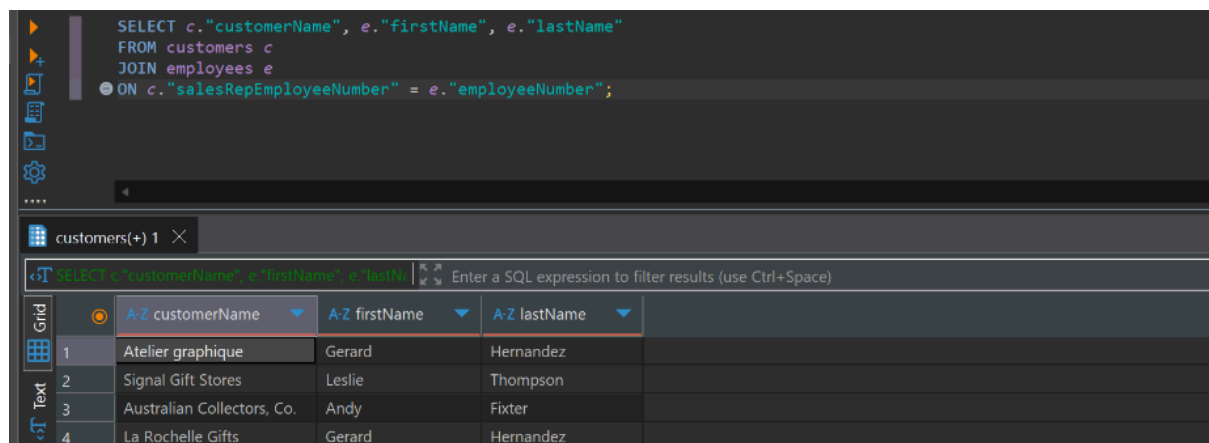
SQL Query

```
SELECT c."customerName", e."firstName", e."lastName"
FROM customers c
JOIN employees e
ON c."salesRepEmployeeNumber" = e."employeeNumber";
```

Explanation

This joins customers with employees to get the assigned sales representative for each customer.

Result



The screenshot shows a database IDE with the following SQL query:

```
SELECT c."customerName", e."firstName", e."lastName"
FROM customers c
JOIN employees e
ON c."salesRepEmployeeNumber" = e."employeeNumber";
```

Below the query editor, a table view titled "customers(+) 1" displays the results. The table has four columns: "A-Z customerName", "A-Z firstName", "A-Z lastName", and an empty column. The results are as follows:

	A-Z customerName	A-Z firstName	A-Z lastName	
1	Atelier graphique	Gerard	Hernandez	
2	Signal Gift Stores	Leslie	Thompson	
3	Australian Collectors, Co.	Andy	Fixter	
4	La Rochelle Gifts	Gerard	Hernandez	

JSON

```
{
  "question": "Get customers with sales rep names",
  "sql": "SELECT c.\"customerName\", e.\"firstName\", e.\"lastName\" FROM
customers c JOIN employees e ON c.\"salesRepEmployeeNumber\" =
e.\"employeeNumber\"",
  "intent": "retrieve customers with their assigned sales representatives",
  "tables": ["customers", "employees"],
  "columns": ["customerName", "firstName", "lastName"],
  "filters": [],
  "joins": ["customers.salesRepEmployeeNumber = employees.employeeNumber"]
}
```

Question 27: Get orders with customer city

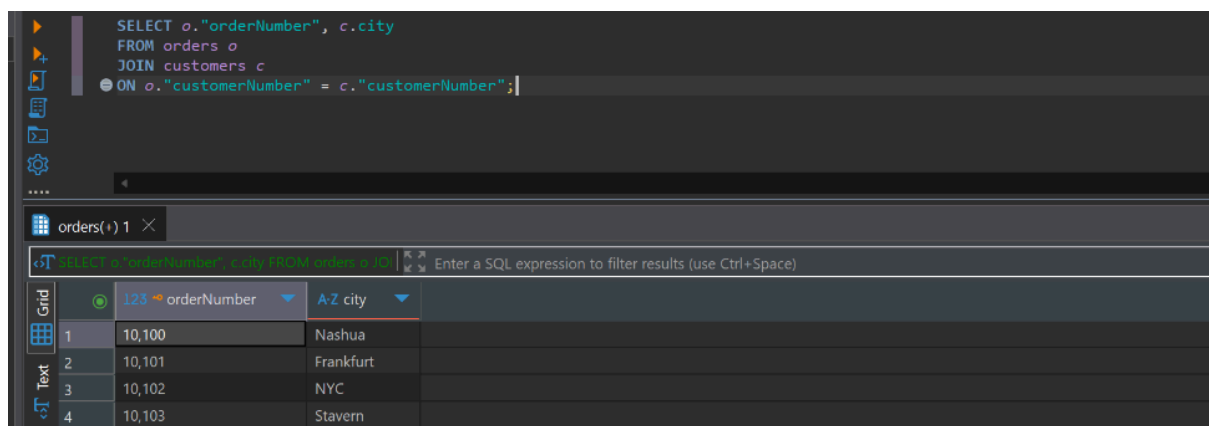
SQL Query

```
SELECT o."orderNumber", c.city
FROM orders o
JOIN customers c
ON o."customerNumber" = c."customerNumber";
```

Explanation

This retrieves orders along with the city of the customer who placed each order.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT o."orderNumber", c.city FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";`. The bottom pane shows the results in a table grid. The table has two columns: `orderNumber` and `city`. The results are as follows:

	orderNumber	city
1	10,100	Nashua
2	10,101	Frankfurt
3	10,102	NYC
4	10,103	Stavern

JSON

```
{
  "question": "Get orders with customer city",
  "sql": "SELECT o.orderNumber, c.city FROM orders o JOIN customers c ON o.customerNumber = c.customerNumber",
  "intent": "retrieve orders with customer location information",
  "tables": ["orders", "customers"],
  "columns": ["orderNumber", "city"],
  "filters": [],
  "joins": ["orders.customerNumber = customers.customerNumber"]
}
```

Question 28: Get employees and their manager

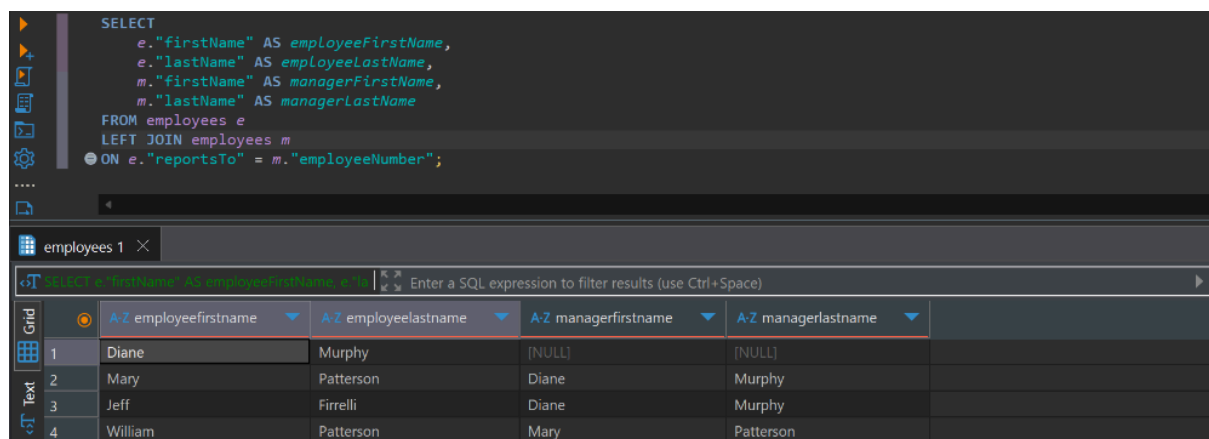
SQL Query

```
SELECT
  e."firstName" AS employeeFirstName,
  e."lastName" AS employeeLastName,
  m."firstName" AS managerFirstName,
  m."lastName" AS managerLastName
FROM employees e
LEFT JOIN employees m
ON e."reportsTo" = m."employeeNumber";
```

Explanation

This performs a self-join on the employees table to find each employee and their manager using the reportsTo field.

Result



The screenshot shows a SQL IDE with the query executed. Below the query editor, a table grid displays the results. The table has five columns: employee first name, employee last name, manager first name, and manager last name. The first row shows Diane Murphy with null manager information. The subsequent rows show employees reporting to Diane Murphy, Jeff Murphy, and Mary Patterson.

	A-Z employeefirstname	A-Z employeelastname	A-Z managerfirstname	A-Z managerlastname
1	Diane	Murphy	[NULL]	[NULL]
2	Mary	Patterson	Diane	Murphy
3	Jeff	Firrelli	Diane	Murphy
4	William	Patterson	Mary	Patterson

JSON

```
{
  "question": "Get employees and their manager",
  "sql": "SELECT e.\"firstName\" AS employeeFirstName, e.\"lastName\" AS employeeLastName, m.\"firstName\" AS managerFirstName, m.\"lastName\" AS managerLastName FROM employees e LEFT JOIN employees m ON e.\"reportsTo\" = m.\"employeeNumber\"",
  "intent": "retrieve employee hierarchy with manager relationships",
  "tables": ["employees"],
  "columns": ["firstName", "lastName", "firstName", "lastName"],
  "filters": [],
  "joins": ["employees.reportsTo = employees.employeeNumber"]
}
```

Question 29: Get orderdetails with product vendor

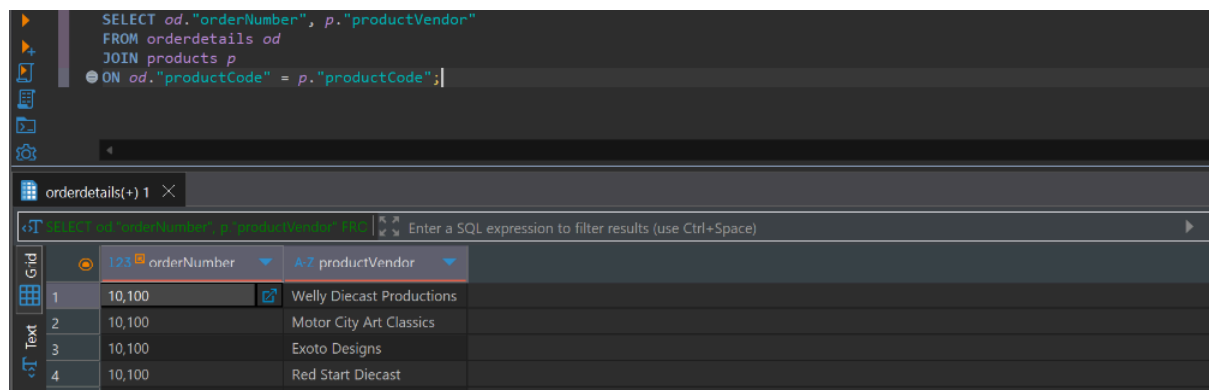
SQL Query

```
SELECT od."orderNumber", p."productVendor"
FROM orderdetails od
JOIN products p
ON od."productCode" = p."productCode";
```

Explanation

This retrieves order details along with product vendor information by joining orderdetails and products.

Result



The screenshot shows a database IDE with the following SQL query in the editor:

```
SELECT od."orderNumber", p."productVendor"
FROM orderdetails od
JOIN products p
ON od."productCode" = p."productCode";
```

Below the editor, the results are displayed in a table view. The table has two columns: 'orderNumber' and 'productVendor'. The results are as follows:

orderNumber	productVendor
10,100	Welly Diecast Productions
10,100	Motor City Art Classics
10,100	Exoto Designs
10,100	Red Start Diecast

JSON

```
{
  "question": "Get orderdetails with product vendor",
  "sql": "SELECT od.\"orderNumber\", p.\"productVendor\" FROM orderdetails od JOIN products p ON od.\"productCode\" = p.\"productCode\"",
  "intent": "retrieve order line items with product vendor information",
  "tables": ["orderdetails", "products"],
  "columns": ["orderNumber", "productVendor"],
  "filters": [],
  "joins": ["orderdetails.productCode = products.productCode"]
}
```

Question 30: Get payments with customer country

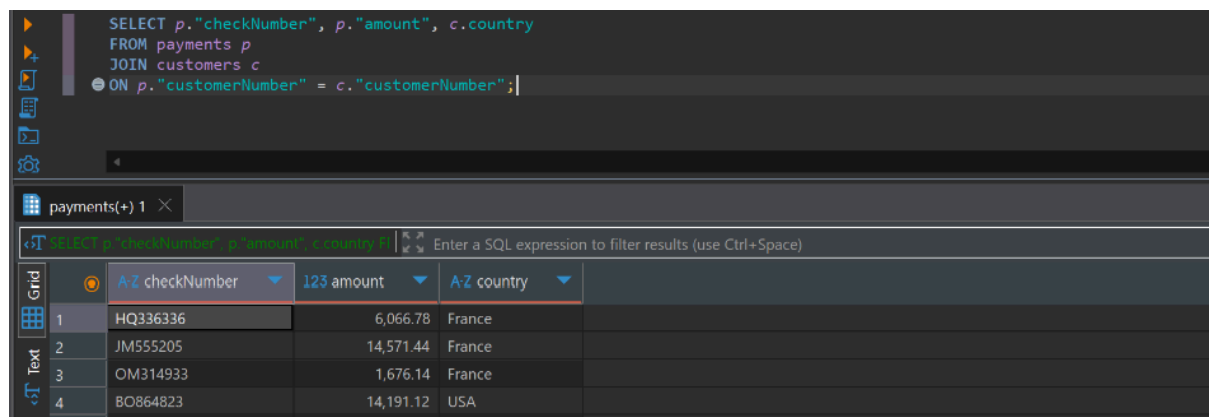
SQL Query

```
SELECT p."checkNumber", p."amount", c.country
FROM payments p
JOIN customers c
ON p."customerNumber" = c."customerNumber";
```

Explanation

This retrieves payments along with the country of the customer who made each payment.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT p."checkNumber", p."amount", c.country FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";`. The bottom pane shows the results in a table view. The table has four columns: `checkNumber`, `amount`, and `country`. There are four rows of data.

	checkNumber	amount	country
1	HQ336336	6,066.78	France
2	JM555205	14,571.44	France
3	OM314933	1,676.14	France
4	BO864823	14,191.12	USA

JSON

```
{
  "question": "Get payments with customer country",
  "sql": "SELECT p.\"checkNumber\", p.\"amount\", c.country FROM payments p JOIN customers c ON p.\"customerNumber\" = c.\"customerNumber\"",
  "intent": "retrieve payments with customer geographic information",
  "tables": ["payments", "customers"],
  "columns": ["checkNumber", "amount", "country"],
  "filters": [],
  "joins": ["payments.customerNumber = customers.customerNumber"]
}
```

Question 31: Count customers per country

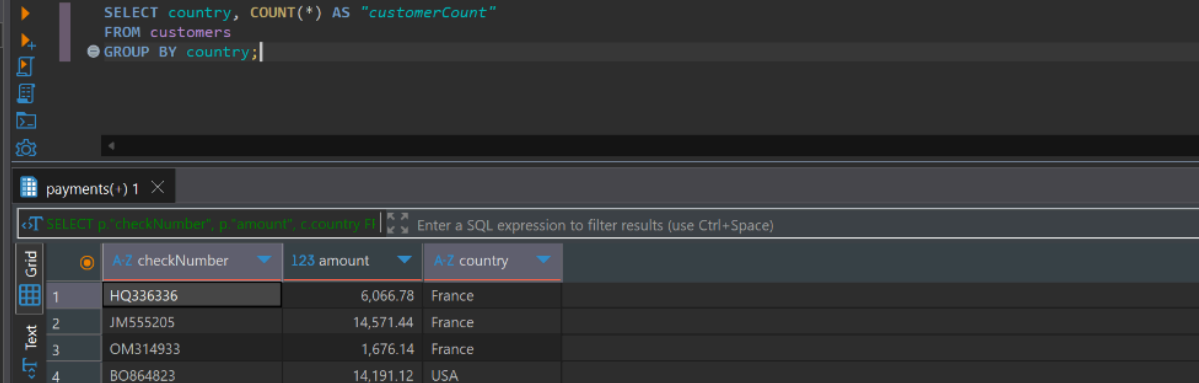
SQL Query

```
SELECT country, COUNT(*) AS "customerCount"
FROM customers
GROUP BY country;
```

Explanation

This counts how many customers exist in each country by grouping customer records by country.

Result



The screenshot shows a SQL IDE interface. The top panel displays the SQL query: `SELECT country, COUNT(*) AS "customerCount" FROM customers GROUP BY country;`. The bottom panel shows the results of the query in a table format. The table has four columns: `checkNumber`, `amount`, and `country`. The results are as follows:

	checkNumber	amount	country
1	HQ336336	6,066.78	France
2	JM555205	14,571.44	France
3	OM314933	1,676.14	France
4	BO864823	14,191.12	USA

JSON

```
{
  "question": "Count customers per country",
  "sql": "SELECT country, COUNT(*) AS \"customerCount\" FROM customers GROUP BY country",
  "intent": "count customers grouped by country",
  "tables": ["customers"],
  "columns": ["country", "customerCount"],
  "filters": [],
  "joins": [],
  "aggregation": ["COUNT"]
}
```

Question 32: Total payments per customer

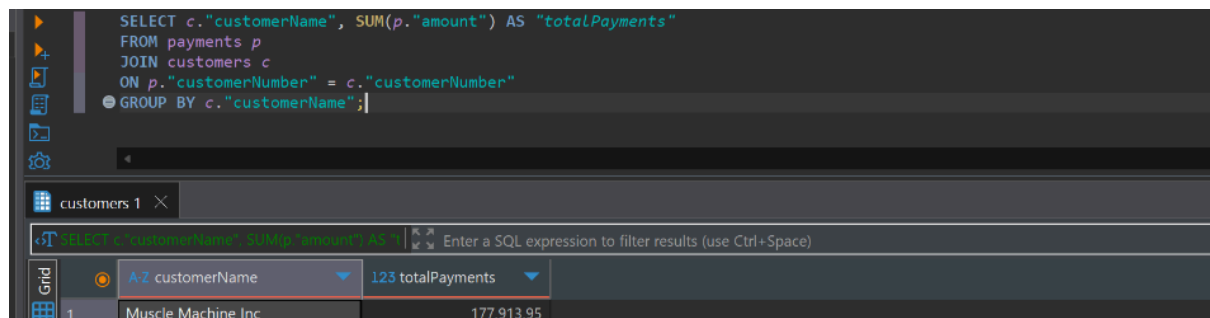
SQL Query

```
SELECT c."customerName", SUM(p."amount") AS "totalPayments"
FROM payments p
JOIN customers c
ON p."customerNumber" = c."customerNumber"
GROUP BY c."customerName";
```

Explanation

This calculates total payment amount made by each customer using SUM aggregation after joining payments with customers.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT c."customerName", SUM(p."amount") AS "totalPayments" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber" GROUP BY c."customerName";`. The bottom pane shows a table with two columns: "customerName" and "totalPayments". The first row contains the values "Muscle Machine Inc" and "177,913.95".

customerName	totalPayments
Muscle Machine Inc	177,913.95

JSON

```
{
  "question": "Total payments per customer",
  "sql": "SELECT c.\"customerName\", SUM(p.\"amount\") AS \"totalPayments\"
FROM payments p JOIN customers c ON p.\"customerNumber\" =
c.\"customerNumber\" GROUP BY c.\"customerName\"",
  "intent": "calculate total payments made by each customer",
  "tables": ["payments", "customers"],
  "columns": ["customerName", "totalPayments"],
  "filters": [],
  "joins": ["payments.customerNumber = customers.customerNumber"],
  "aggregation": ["SUM"]
}
```


Question 33: Number of orders per status

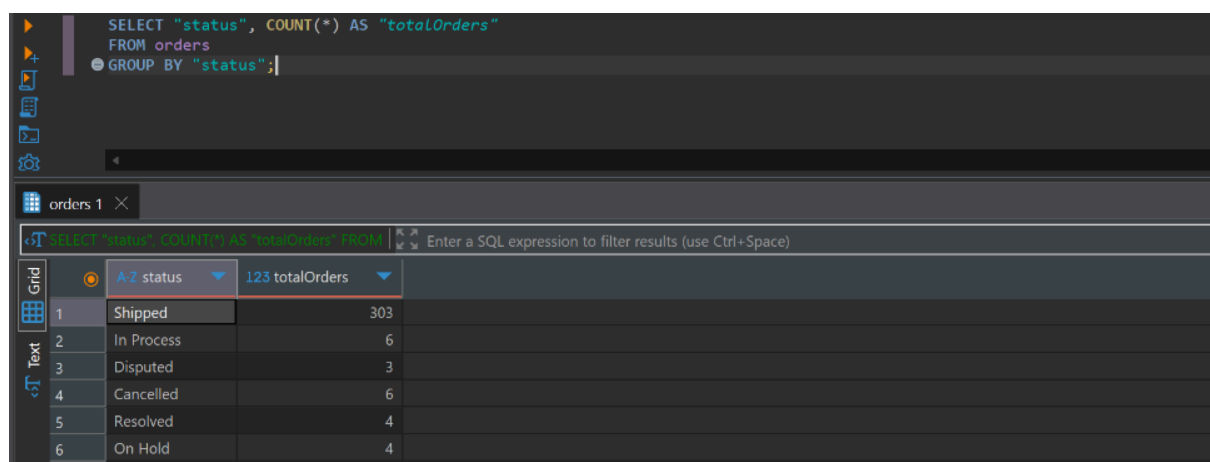
SQL Query

```
SELECT "status", COUNT(*) AS "totalOrders"
FROM orders
GROUP BY "status";
```

Explanation

This counts how many orders exist for each order status (e.g., Shipped, In Process).

Result



The screenshot shows a SQL IDE interface. The top panel displays the SQL query: `SELECT "status", COUNT(*) AS "totalOrders" FROM orders GROUP BY "status";`. Below the query editor, a table view titled "orders 1" shows the results. The table has two columns: "status" and "totalOrders". The results are as follows:

status	totalOrders
Shipped	303
In Process	6
Disputed	3
Cancelled	6
Resolved	4
On Hold	4

JSON

```
{
  "question": "Number of orders per status",
  "sql": "SELECT \"status\", COUNT(*) AS \"totalOrders\" FROM orders GROUP BY \"status\"",
  "intent": "count orders grouped by status",
  "tables": ["orders"],
  "columns": ["status", "totalOrders"],
  "filters": [],
  "joins": [],
  "aggregation": ["COUNT"]
}
```

Question 34: Products per product line

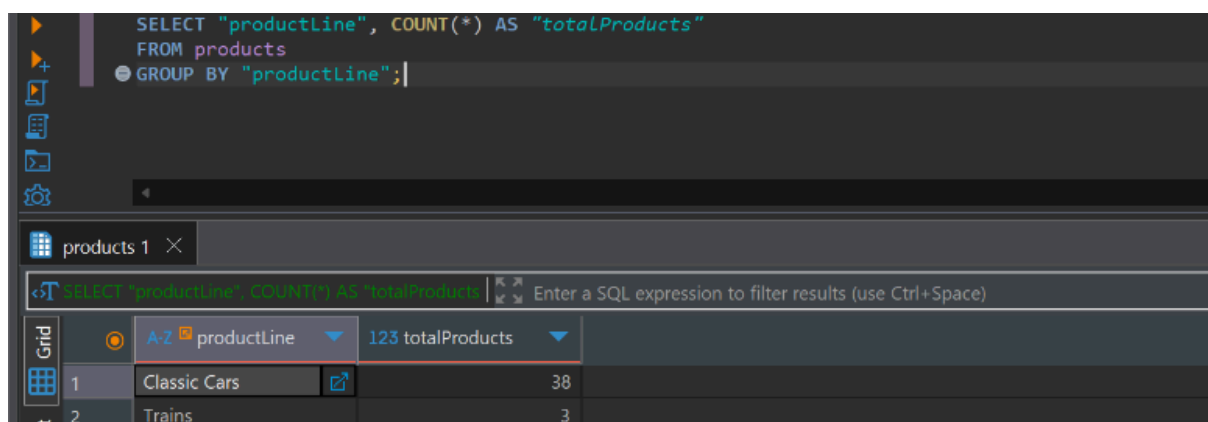
SQL Query

```
SELECT "productLine", COUNT(*) AS "totalProducts"
FROM products
GROUP BY "productLine";
```

Explanation

This counts how many products exist in each product line category.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT "productLine", COUNT(*) AS "totalProducts" FROM products GROUP BY "productLine";`. The bottom pane shows the results in a table view. The table has two columns: "productLine" and "totalProducts". There are two rows of data: "Classic Cars" with a count of 38, and "Trains" with a count of 3.

	productLine	totalProducts
1	Classic Cars	38
2	Trains	3

JSON

```
{
  "question": "Products per product line",
  "sql": "SELECT \"productLine\", COUNT(*) AS \"totalProducts\" FROM products\nGROUP BY \"productLine\"",
  "intent": "count products grouped by product line",
  "tables": ["products"],
  "columns": ["productLine", "totalProducts"],
  "filters": [],
  "joins": [],
  "aggregation": ["COUNT"]
}
```

Question 35: Employees per office

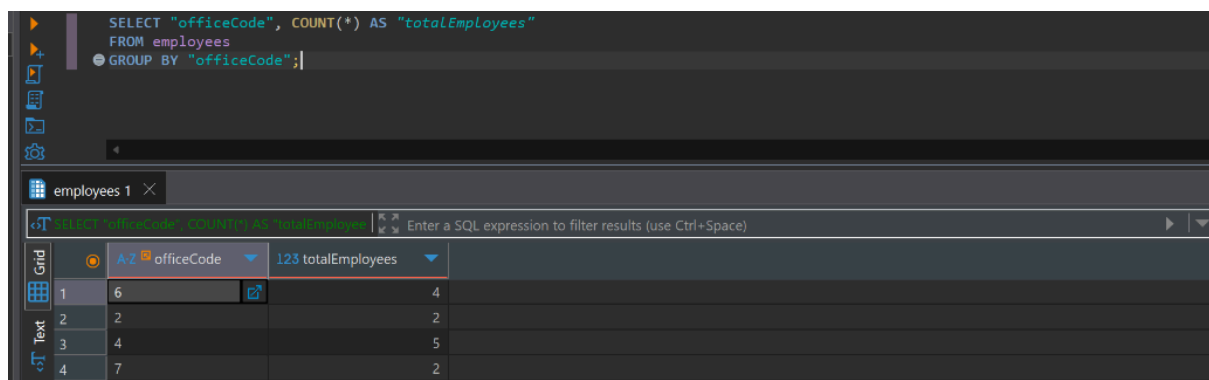
SQL Query

```
SELECT "officeCode", COUNT(*) AS "totalEmployees"  
FROM employees  
GROUP BY "officeCode";
```

Explanation

This counts how many employees are assigned to each office.

Result



The screenshot shows a SQL IDE with the query: `SELECT "officeCode", COUNT(*) AS "totalEmployees" FROM employees GROUP BY "officeCode";`. Below the query editor, a table titled "employees 1" displays the results. The table has two columns: "officeCode" and "totalEmployees". The results are as follows:

Grid	officeCode	totalEmployees
1	6	4
2	2	2
3	4	5
4	7	2

JSON

```
{  
  "question": "Employees per office",  
  "sql": "SELECT \"officeCode\", COUNT(*) AS \"totalEmployees\" FROM employees  
GROUP BY \"officeCode\"",  
  "intent": "count employees grouped by office",  
  "tables": ["employees"],  
  "columns": ["officeCode", "totalEmployees"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["COUNT"]  
}
```

Question 36: Total stock per product vendor

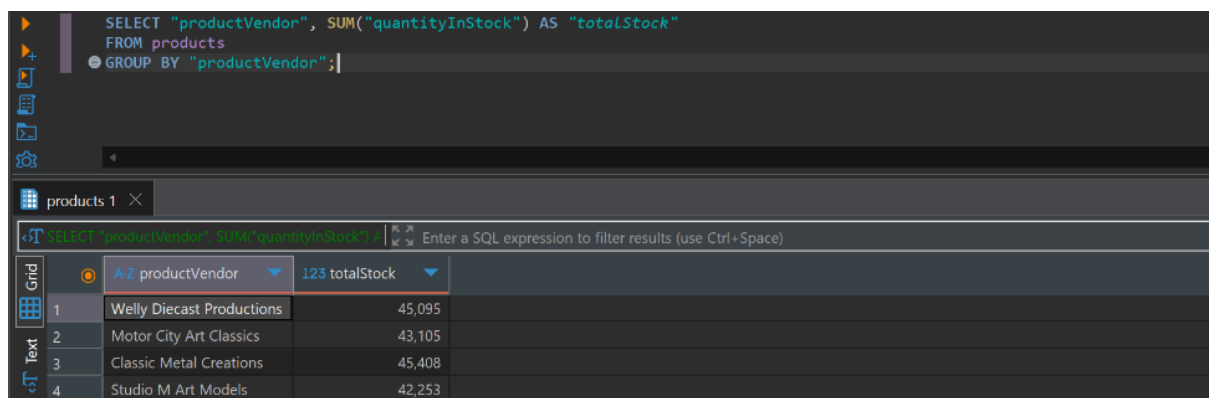
SQL Query

```
SELECT "productVendor", SUM("quantityInStock") AS "totalStock"
FROM products
GROUP BY "productVendor";
```

Explanation

This calculates total inventory stock grouped by product vendor.

Result



The screenshot shows a SQL IDE interface. The top panel displays the SQL query: `SELECT "productVendor", SUM("quantityInStock") AS "totalStock" FROM products GROUP BY "productVendor";`. Below the query editor, a table grid displays the results. The table has two columns: "productVendor" and "totalStock". The results are as follows:

	productVendor	totalStock
1	Welly Diecast Productions	45,095
2	Motor City Art Classics	43,105
3	Classic Metal Creations	45,408
4	Studio M Art Models	42,253

JSON

```
{
  "question": "Total stock per product vendor",
  "sql": "SELECT \"productVendor\", SUM(\"quantityInStock\") AS \"totalStock\" FROM products GROUP BY \"productVendor\"",
  "intent": "calculate total inventory stock grouped by vendor",
  "tables": ["products"],
  "columns": ["productVendor", "totalStock"],
  "filters": [],
  "joins": [],
  "aggregation": ["SUM"]
}
```

Question 37: Average buy price per product line

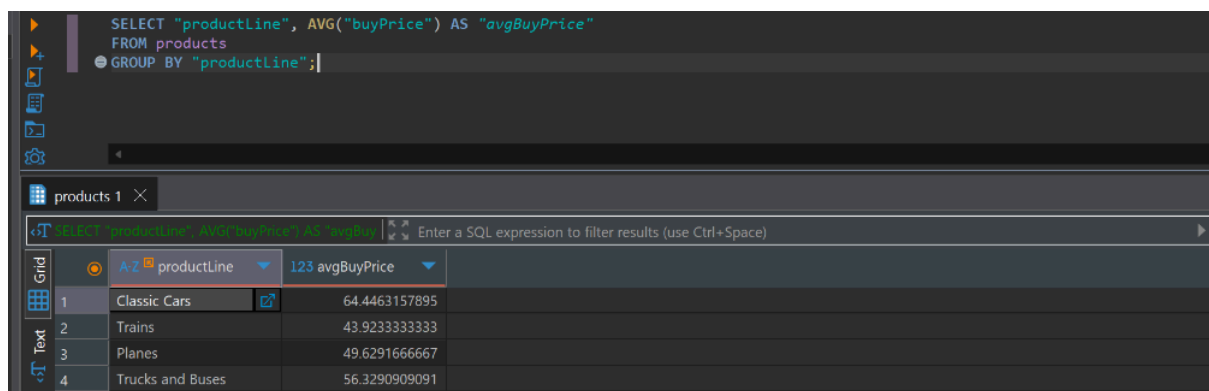
SQL Query

```
SELECT "productLine", AVG("buyPrice") AS "avgBuyPrice"
FROM products
GROUP BY "productLine";
```

Explanation

This calculates the average buying price of products in each product line.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT "productLine", AVG("buyPrice") AS "avgBuyPrice" FROM products GROUP BY "productLine";`. The bottom pane shows the results in a table with two columns: `productLine` and `avgBuyPrice`. The results are as follows:

productLine	avgBuyPrice
Classic Cars	64.4463157895
Trains	43.9233333333
Planes	49.6291666667
Trucks and Buses	56.3290909091

JSON

```
{
  "question": "Average buy price per product line",
  "sql": "SELECT \"productLine\", AVG(\"buyPrice\") AS \"avgBuyPrice\" FROM products GROUP BY \"productLine\"",
  "intent": "calculate average product cost grouped by product line",
  "tables": ["products"],
  "columns": ["productLine", "avgBuyPrice"],
  "filters": [],
  "joins": [],
  "aggregation": ["AVG"]
}
```

Question 38: Orders per customer

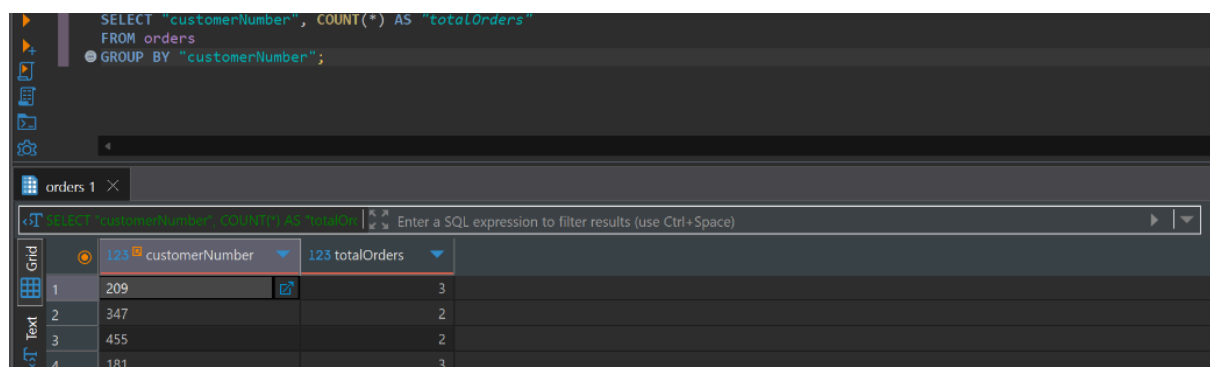
SQL Query

```
SELECT "customerNumber", COUNT(*) AS "totalOrders"  
FROM orders  
GROUP BY "customerNumber";
```

Explanation

This counts total orders placed by each customer.

Result



The screenshot shows a SQL IDE interface. At the top, the SQL query is entered in a text area: `SELECT "customerNumber", COUNT(*) AS "totalOrders" FROM orders GROUP BY "customerNumber";`. Below the query, a tab labeled "orders 1" is active. Underneath the tab, there is a filter bar with the same query text and a search icon. Below the filter bar, a table with two columns, "customerNumber" and "totalOrders", displays the results. The table has four rows of data.

	customerNumber	totalOrders
1	209	3
2	347	2
3	455	2
4	181	3

JSON

```
{  
  
  "question": "Orders per customer",  
  
  "sql": "SELECT \"customerNumber\", COUNT(*) AS \"totalOrders\" FROM orders  
GROUP BY \"customerNumber\"",  
  
  "intent": "count orders grouped by customer",  
  
  "tables": ["orders"],  
  
  "columns": ["customerNumber", "totalOrders"],  
  
  "filters": [],  
  
  "joins": [],  
  
  "aggregation": ["COUNT"]  
}
```

Question 39: Max MSRP per product line

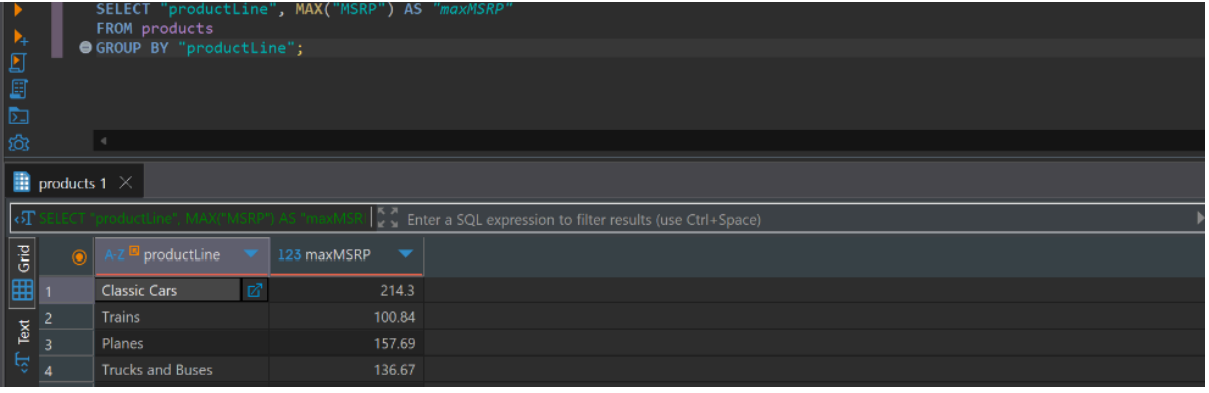
SQL Query

```
SELECT "productLine", MAX("MSRP") AS "maxMSRP"  
FROM products  
GROUP BY "productLine";
```

Explanation

This finds the highest MSRP value for products in each product line.

Result



The screenshot shows a SQL IDE interface. The top pane contains the SQL query: `SELECT "productLine", MAX("MSRP") AS "maxMSRP" FROM products GROUP BY "productLine";`. The bottom pane shows a table grid with the results of the query. The table has two columns: "productLine" and "maxMSRP". The results are as follows:

	productLine	maxMSRP
1	Classic Cars	214.3
2	Trains	100.84
3	Planes	157.69
4	Trucks and Buses	136.67

JSON

```
{  
  
  "question": "Max MSRP per product line",  
  
  "sql": "SELECT \"productLine\", MAX(\"MSRP\") AS \"maxMSRP\" FROM products  
GROUP BY \"productLine\",  
  
  "intent": "find maximum MSRP grouped by product line",  
  
  "tables": ["products"],  
  
  "columns": ["productLine", "maxMSRP"],  
  
  "filters": [],  
  
  "joins": [],  
  
  "aggregation": ["MAX"]  
}
```

Question 40: Min buy price per vendor

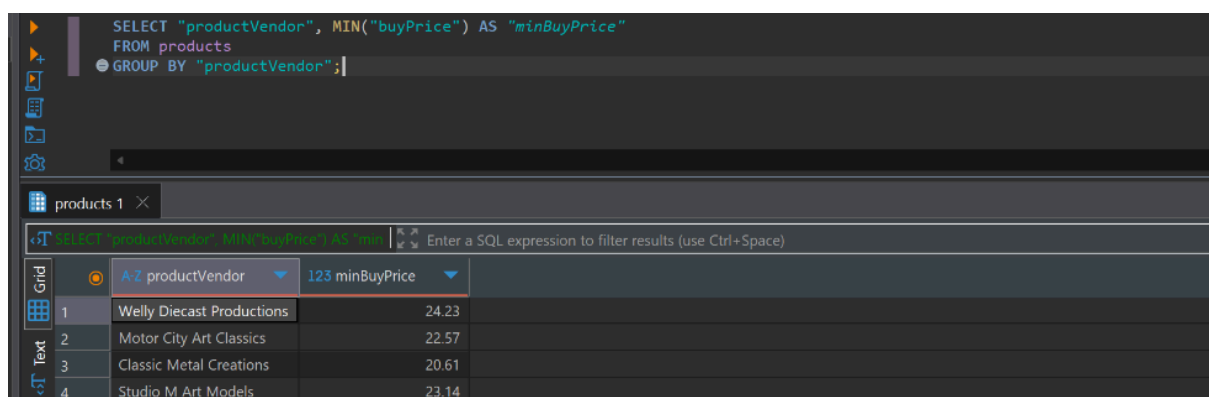
SQL Query

```
SELECT "productVendor", MIN("buyPrice") AS "minBuyPrice"
FROM products
GROUP BY "productVendor";
```

Explanation

This finds the lowest buy price for products grouped by vendor.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT "productVendor", MIN("buyPrice") AS "minBuyPrice" FROM products GROUP BY "productVendor";`. The bottom pane shows the results in a table grid. The table has two columns: "productVendor" and "minBuyPrice". There are four rows of data.

	productVendor	minBuyPrice
1	Welly Diecast Productions	24.23
2	Motor City Art Classics	22.57
3	Classic Metal Creations	20.61
4	Studio M Art Models	23.14

JSON

```
{
  "question": "Min buy price per vendor",
  "sql": "SELECT \"productVendor\", MIN(\"buyPrice\") AS \"minBuyPrice\" FROM products GROUP BY \"productVendor\"",
  "intent": "find minimum product cost grouped by vendor",
  "tables": ["products"],
  "columns": ["productVendor", "minBuyPrice"],
  "filters": [],
  "joins": [],
  "aggregation": ["MIN"]
}
```


Question 41: Total number of customers

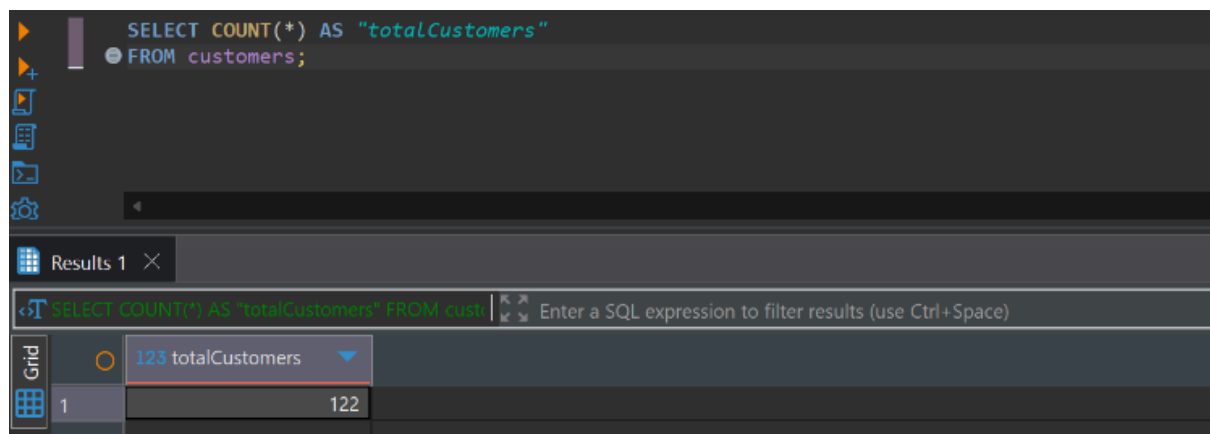
SQL Query

```
SELECT COUNT(*) AS "totalCustomers"  
FROM customers;
```

Explanation

This returns the total number of customer records in the database.

Result



The screenshot shows a SQL IDE interface. The top pane contains the query: `SELECT COUNT(*) AS "totalCustomers" FROM customers;`. The bottom pane shows the results in a table grid. The grid has one column named `totalCustomers` and one row with the value `122`.

totalCustomers
122

JSON

```
{  
  "question": "Total number of customers",  
  "sql": "SELECT COUNT(*) AS \"totalCustomers\" FROM customers",  
  "intent": "count total customers",  
  "tables": ["customers"],  
  "columns": ["totalCustomers"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["COUNT"]  
}
```

Question 42: Total number of products

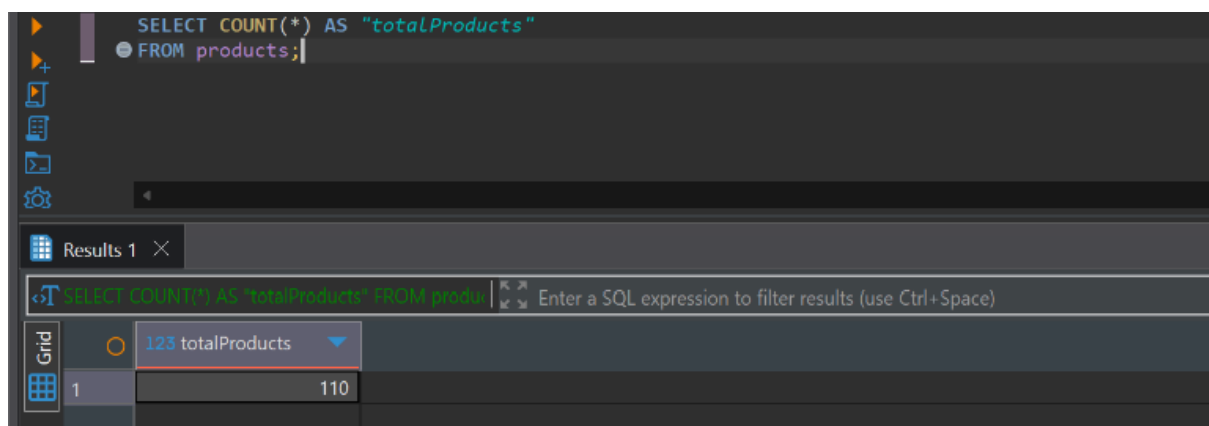
SQL Query

```
SELECT COUNT(*) AS "totalProducts"  
FROM products;
```

Explanation

This counts the total number of products in the system.

Result



JSON

```
{  
  "question": "Total number of products",  
  "sql": "SELECT COUNT(*) AS \"totalProducts\" FROM products",  
  "intent": "count total products",  
  "tables": ["products"],  
  "columns": ["totalProducts"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["COUNT"]  
}
```

Question 43: Total revenue from payments

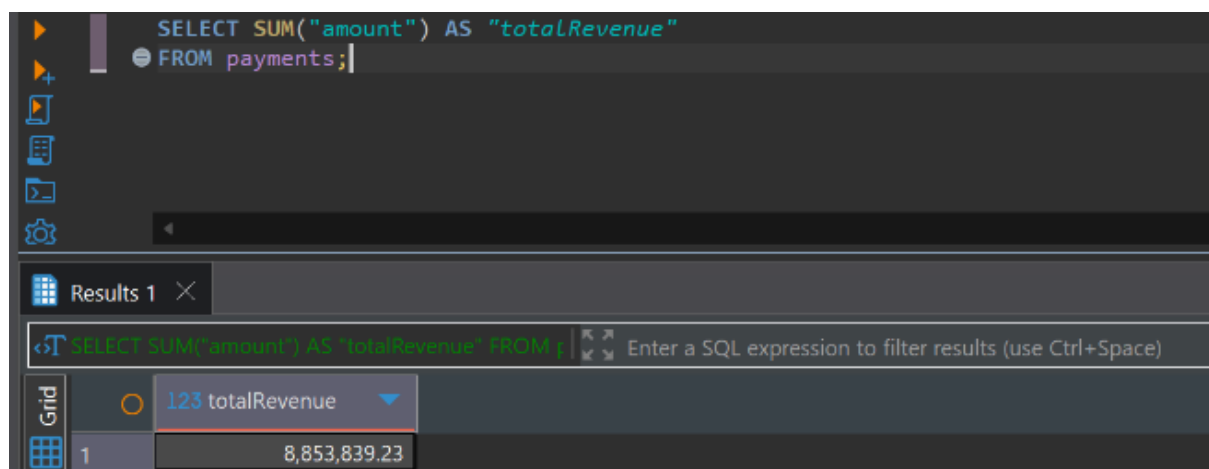
SQL Query

```
SELECT SUM("amount") AS "totalRevenue"  
FROM payments;
```

Explanation

This calculates total revenue collected from all payments.

Result



The screenshot shows a SQL query editor with the following query:

```
SELECT SUM("amount") AS "totalRevenue"  
FROM payments;
```

Below the query editor, the results are displayed in a grid view. The grid has one column named "totalRevenue" and one row with the value 8,853,839.23.

totalRevenue
8,853,839.23

JSON

```
{  
  "question": "Total revenue from payments",  
  "sql": "SELECT SUM(\"amount\") AS \"totalRevenue\" FROM payments",  
  "intent": "calculate total revenue from payments",  
  "tables": ["payments"],  
  "columns": ["totalRevenue"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["SUM"]  
}
```

Question 44: Average product price

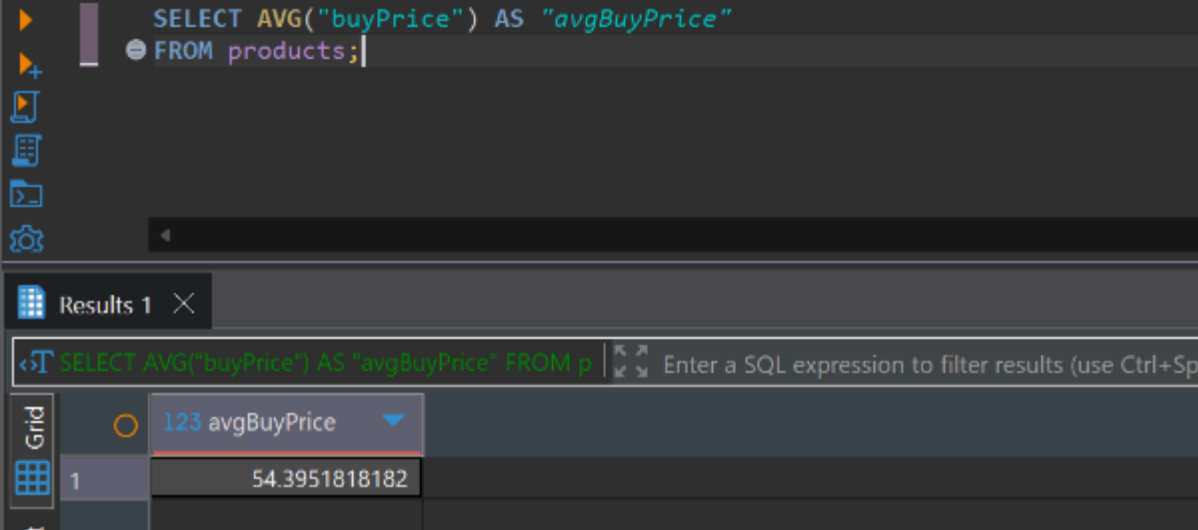
SQL Query

```
SELECT AVG("buyPrice") AS "avgBuyPrice"  
FROM products;
```

Explanation

This computes the average buy price across all products.

Result



The screenshot shows a SQL IDE interface. The top pane displays the query: `SELECT AVG("buyPrice") AS "avgBuyPrice" FROM products;`. The bottom pane, titled "Results 1", shows the result of the query in a table format. The table has one column, "avgBuyPrice", and one row with the value 54.3951818182.

avgBuyPrice
54.3951818182

JSON

```
{  
  "question": "Total revenue from payments",  
  "sql": "SELECT SUM(\"amount\") AS \"totalRevenue\" FROM payments",  
  "intent": "calculate total revenue from payments",  
  "tables": ["payments"],  
  "columns": ["totalRevenue"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["SUM"]  
}
```

Question 45: Max payment amount

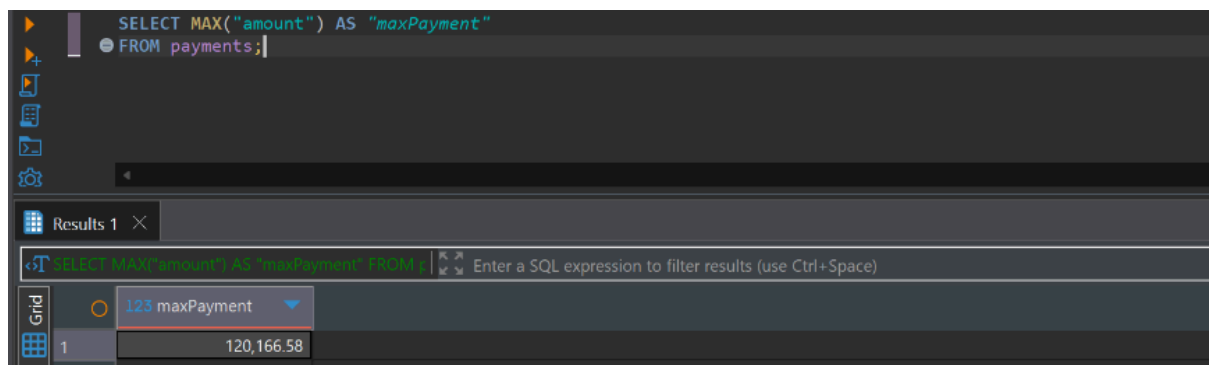
SQL Query

```
SELECT MAX("amount") AS "maxPayment"  
FROM payments;
```

Explanation

This finds the highest payment amount in the payments table.

Result



The screenshot shows a SQL IDE interface. The top pane displays the query: `SELECT MAX("amount") AS "maxPayment" FROM payments;`. The bottom pane shows the results in a table grid. The first column is labeled "maxPayment" and the first row contains the value "120,166.58".

maxPayment
120,166.58

JSON

```
{  
  "question": "Max payment amount",  
  "sql": "SELECT MAX(\"amount\") AS \"maxPayment\" FROM payments",  
  "intent": "find maximum payment amount",  
  "tables": ["payments"],  
  "columns": ["maxPayment"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["MAX"]  
}
```

Question 46: Min payment amount

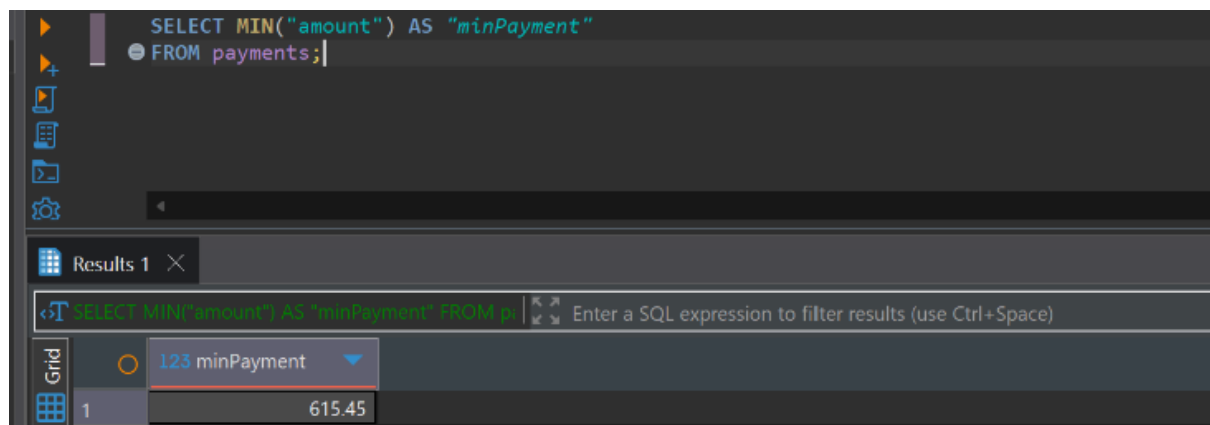
SQL Query

```
SELECT MIN("amount") AS "minPayment"  
FROM payments;
```

Explanation

This finds the smallest payment amount in the payments table.

Result



The screenshot shows a SQL IDE interface. The top pane displays the query: `SELECT MIN("amount") AS "minPayment" FROM payments;`. The bottom pane shows the results in a table with one column, `minPayment`, and one row with the value `615.45`.

minPayment
615.45

JSON

```
{  
  "question": "Min payment amount",  
  "sql": "SELECT MIN(\"amount\") AS \"minPayment\" FROM payments",  
  "intent": "find minimum payment amount",  
  "tables": ["payments"],  
  "columns": ["minPayment"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["MIN"]  
}
```

Question 47: Count total orders

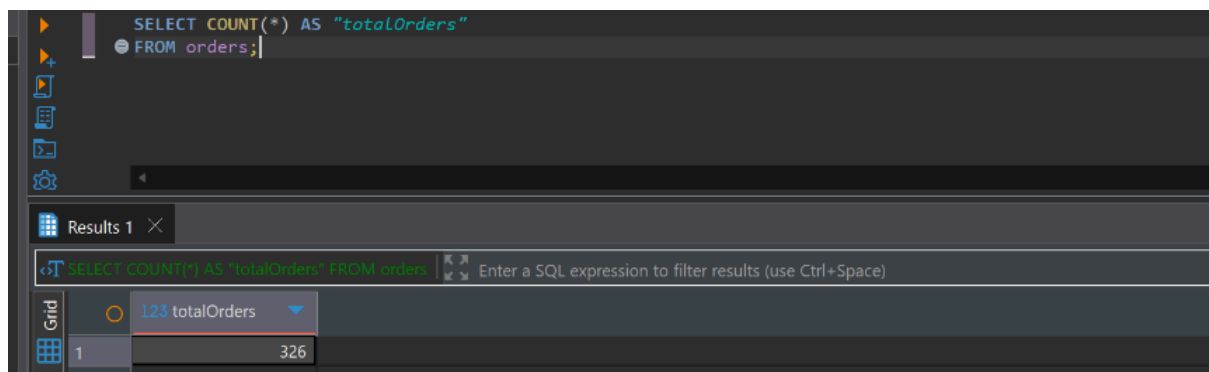
SQL Query

```
SELECT COUNT(*) AS "totalOrders"  
FROM orders;
```

Explanation

This counts all orders in the system.

Result



The screenshot shows a SQL IDE interface. The top pane contains the SQL query: `SELECT COUNT(*) AS "totalOrders" FROM orders;`. The bottom pane, titled "Results 1", shows the query result in a grid view. The grid has one column named "totalOrders" and one row with the value 326.

totalOrders
326

JSON

```
{  
  
  "question": "Count total orders",  
  
  "sql": "SELECT COUNT(*) AS \"totalOrders\" FROM orders",  
  
  "intent": "count total orders",  
  
  "tables": ["orders"],  
  
  "columns": ["totalOrders"],  
  
  "filters": [],  
  
  "joins": [],  
  
  "aggregation": ["COUNT"]  
}
```

Question 48: Total quantity in stock

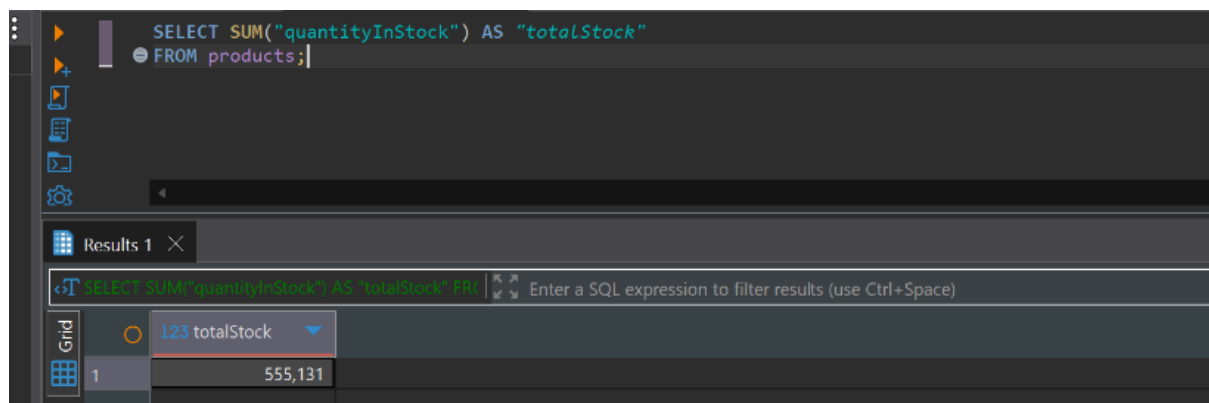
SQL Query

```
SELECT SUM("quantityInStock") AS "totalStock"  
FROM products;
```

Explanation

This calculates total inventory across all products.

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT SUM("quantityInStock") AS "totalStock" FROM products;`. The bottom pane shows the results in a table format. The table has one column named `totalStock` and one row with the value `555,131`.

totalStock
555,131

JSON

```
{  
  "question": "Total quantity in stock",  
  "sql": "SELECT SUM(\"quantityInStock\") AS \"totalStock\" FROM products",  
  "intent": "calculate total inventory stock",  
  "tables": ["products"],  
  "columns": ["totalStock"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["SUM"]  
}
```


Question 49: Average MSRP

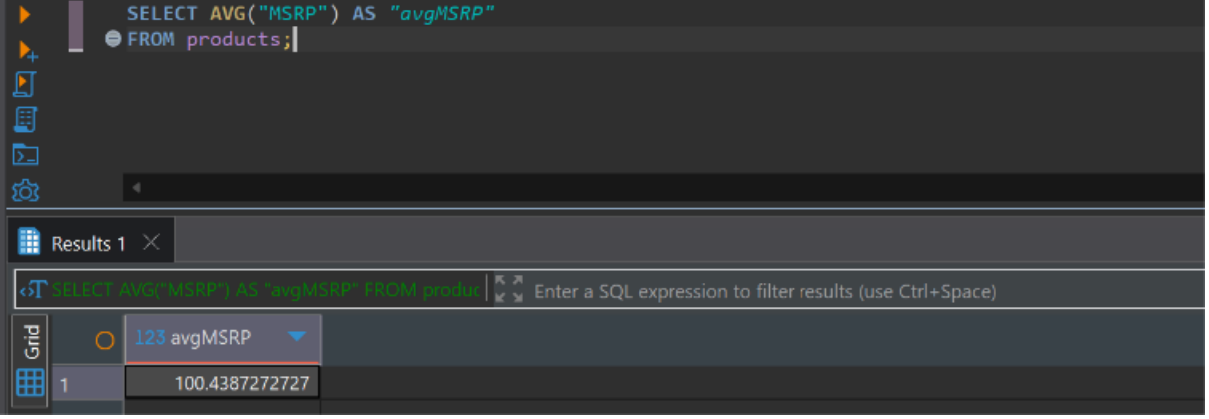
SQL Query

```
SELECT AVG("MSRP") AS "avgMSRP"  
FROM products;
```

Explanation

This computes the average MSRP value of all products.

Result



The screenshot shows a SQL IDE interface. The top pane displays the query: `SELECT AVG("MSRP") AS "avgMSRP" FROM products;`. The bottom pane shows the results in a table grid. The first column is labeled '1' and the second column is labeled 'avgMSRP'. The value in the first row is 100.4387272727.

1	avgMSRP
1	100.4387272727

JSON

```
{  
  "question": "Average MSRP",  
  "sql": "SELECT AVG(\"MSRP\") AS \"avgMSRP\" FROM products",  
  "intent": "calculate average MSRP value",  
  "tables": ["products"],  
  "columns": ["avgMSRP"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["AVG"]  
}
```

Question 50: Number of employees

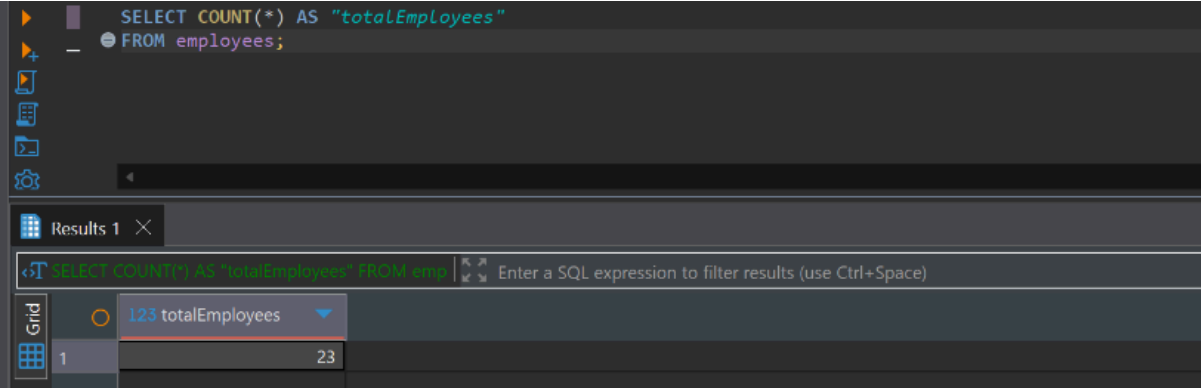
SQL Query

```
SELECT COUNT(*) AS "totalEmployees"  
FROM employees;
```

Explanation

This counts total number of employees in the system.

Result



The screenshot shows a SQL IDE interface. The top pane displays the query: `SELECT COUNT(*) AS "totalEmployees" FROM employees;`. The bottom pane shows the results in a table format. The table has one column named `totalEmployees` and one row with the value `23`.

totalEmployees
23

JSON

```
{  
  "question": "Number of employees",  
  "sql": "SELECT COUNT(*) AS \"totalEmployees\" FROM employees",  
  "intent": "count total employees",  
  "tables": ["employees"],  
  "columns": ["totalEmployees"],  
  "filters": [],  
  "joins": [],  
  "aggregation": ["COUNT"]  
}
```

ADDITIONAL QUESTIONS FOR DATASET

Question 51: Customers from USA

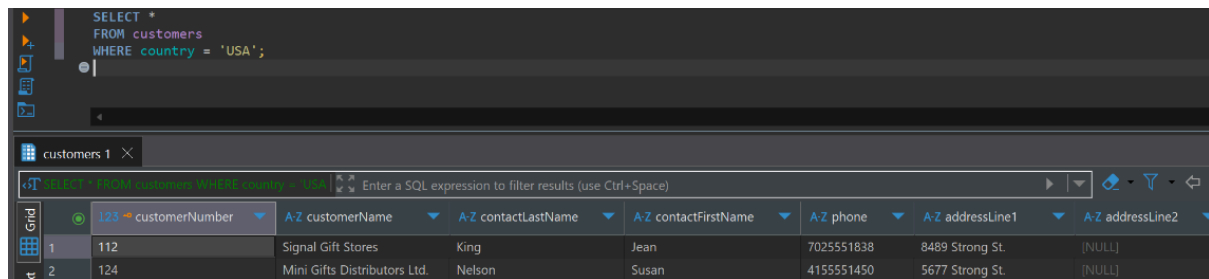
SQL Query

```
SELECT *  
FROM customers  
WHERE country = 'USA';
```

Explanation

This filters customers and returns only those whose country is USA.

Result



The screenshot shows a SQL query editor with the query: `SELECT * FROM customers WHERE country = 'USA';`. Below the query, a table titled "customers 1" displays the results. The table has 8 columns: customerNumber, customerName, contactLastName, contactFirstName, phone, addressLine1, and addressLine2. There are 2 rows of data.

	customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1	addressLine2
1	112	Signal Gift Stores	King	Jean	7025551838	8489 Strong St.	[NULL]
2	124	Mini Gifts Distributors Ltd.	Nelson	Susan	4155551450	5677 Strong St.	[NULL]

JSON

```
{  
  
  "question": "Customers from USA",  
  
  "sql": "SELECT * FROM customers WHERE country = 'USA'",  
  
  "intent": "filter customers by country",  
  
  "tables": ["customers"],  
  
  "columns": ["*"],  
  
  "filters": ["country = USA"],  
  
  "joins": [],  
  
  "aggregation": []  
}
```

Question 52: Orders placed after 2005-01-01

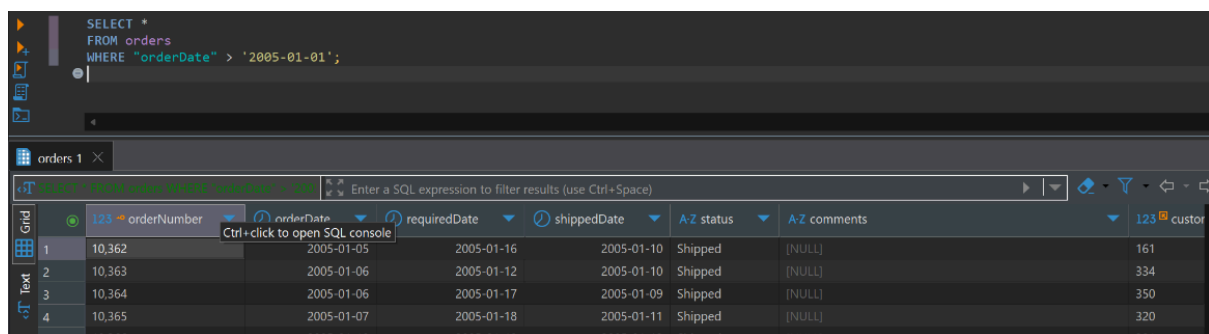
SQL Query

```
SELECT *  
FROM orders  
WHERE "orderDate" > '2005-01-01';
```

Explanation

This retrieves orders that were placed after January 1, 2005.

Result



The screenshot shows a SQL query editor with the following query:

```
SELECT *  
FROM orders  
WHERE "orderDate" > '2005-01-01';
```

The results are displayed in a table with the following columns: orderNumber, orderDate, requiredDate, shippedDate, status, comments, and customer. The table contains 4 rows of data.

	orderNumber	orderDate	requiredDate	shippedDate	status	comments	customer
1	10362	2005-01-05	2005-01-16	2005-01-10	Shipped	[NULL]	161
2	10363	2005-01-06	2005-01-12	2005-01-10	Shipped	[NULL]	334
3	10364	2005-01-06	2005-01-17	2005-01-09	Shipped	[NULL]	350
4	10365	2005-01-07	2005-01-18	2005-01-11	Shipped	[NULL]	320

JSON

```
{  
  "question": "Orders placed after 2005-01-01",  
  "sql": "SELECT * FROM orders WHERE \"orderDate\" > '2005-01-01'",  
  "intent": "filter orders by date",  
  "tables": ["orders"],  
  "columns": ["*"],  
  "filters": ["orderDate > 2005-01-01"],  
  "joins": [],  
  "aggregation": []  
}
```

Question 53: Products with buy price greater than 100

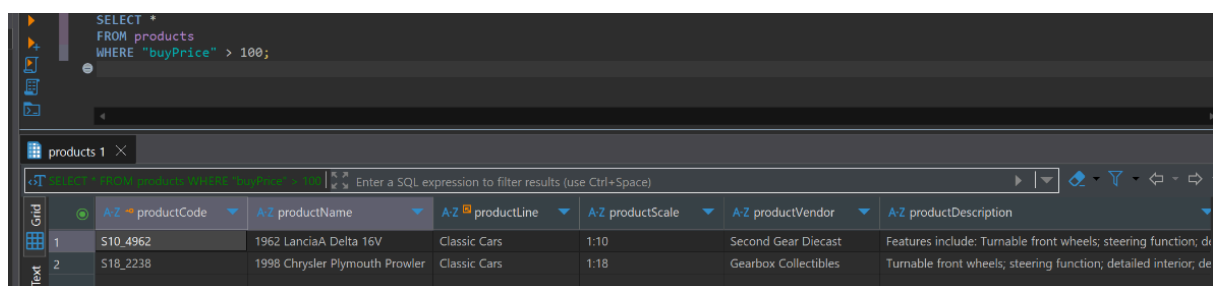
SQL Query

```
SELECT *  
FROM products  
WHERE "buyPrice" > 100;
```

Explanation

This filters products where the buy price is greater than 100.

Result



The screenshot shows a database query tool interface. At the top, the SQL query is displayed: `SELECT * FROM products WHERE "buyPrice" > 100;`. Below the query, a table titled "products 1" shows the results. The table has 7 columns: productCode, productName, productLine, productScale, productVendor, and productDescription. Two rows are visible, both with product codes starting with 'S10'.

	productCode	productName	productLine	productScale	productVendor	productDescription
1	S10_4962	1962 LanciaA Delta 16V	Classic Cars	1:10	Second Gear Diecast	Features include: Turnable front wheels; steering function; d
2	S10_2238	1998 Chrysler Plymouth Prowler	Classic Cars	1:18	Gearbox Collectibles	Turnable front wheels; steering function, detailed interior, de

JSON

```
{  
  "question": "Products with buy price greater than 100",  
  "sql": "SELECT * FROM products WHERE \"buyPrice\" > 100",  
  "intent": "filter products by price",  
  "tables": ["products"],  
  "columns": ["*"],  
  "filters": ["buyPrice > 100"],  
  "joins": [],  
  "aggregation": []  
}
```

Question 54: Payments greater than 5000

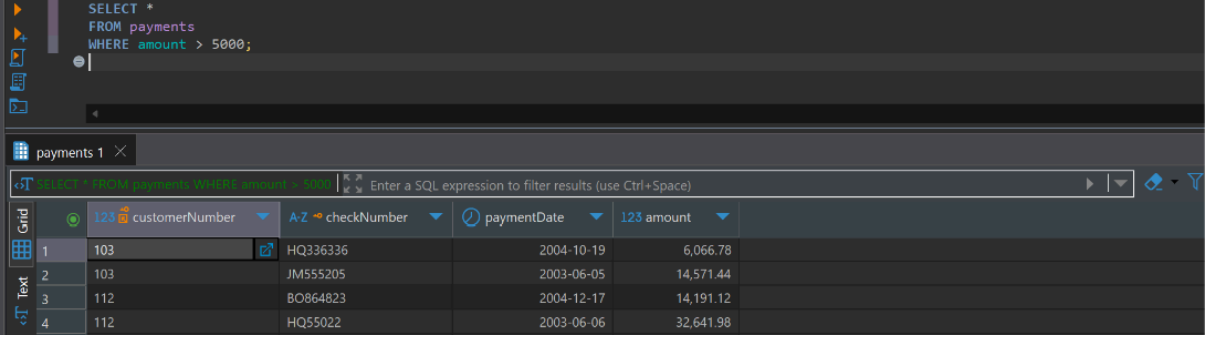
SQL Query

```
SELECT *  
FROM payments  
WHERE amount > 5000;
```

Explanation

This retrieves all payments where amount is greater than 5000.

Result



The screenshot shows a database query tool interface. At the top, the SQL query is entered: `SELECT * FROM payments WHERE amount > 5000;`. Below the query editor, a table titled "payments 1" displays the results. The table has five columns: an index, customerNumber, checkNumber, paymentDate, and amount. There are four rows of data shown.

	customerNumber	checkNumber	paymentDate	amount
1	103	HQ336336	2004-10-19	6,066.78
2	103	JM555205	2003-06-05	14,571.44
3	112	BO864823	2004-12-17	14,191.12
4	112	HQ55022	2003-06-06	32,641.98

JSON

```
{  
  "question": "Payments greater than 5000",  
  "sql": "SELECT * FROM payments WHERE amount > 5000",  
  "intent": "filter payments by amount threshold",  
  "tables": ["payments"],  
  "columns": ["*"],  
  "filters": ["amount > 5000"],  
  "joins": [],  
  "aggregation": []  
}
```

Question 55: Employees from office code 1

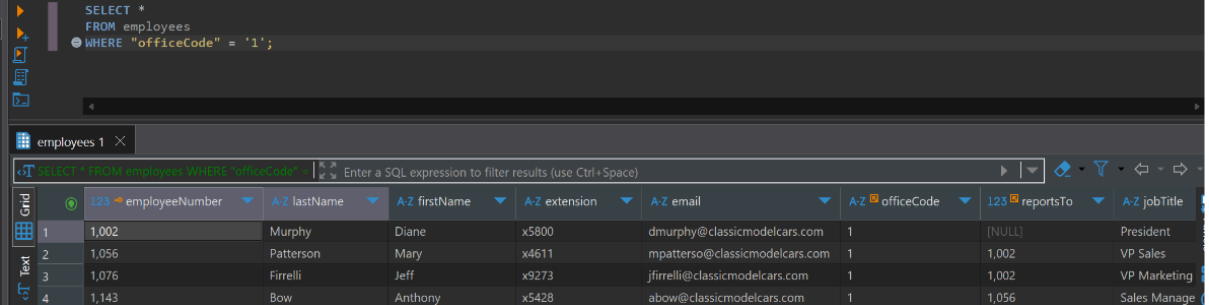
SQL Query

```
SELECT *  
FROM employees  
WHERE "officeCode" = '1';
```

Explanation

This filters employees working in office code 1.

Result



The screenshot shows a SQL IDE with the query `SELECT * FROM employees WHERE "officeCode" = '1';` in the top pane. The bottom pane displays a table grid with the following data:

	123 employeeNumber	A-Z lastName	A-Z firstName	A-Z extension	A-Z email	A-Z officeCode	123 reportsTo	A-Z jobTitle
1	1,002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	[NULL]	President
2	1,056	Patterson	Mary	x4611	mpatterso@classicmodelcars.com	1	1,002	VP Sales
3	1,076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1,002	VP Marketing
4	1,143	Bow	Anthony	x5428	abow@classicmodelcars.com	1	1,056	Sales Manage

JSON

```
{  
  
  "question": "Employees from office code 1",  
  
  "sql": "SELECT * FROM employees WHERE \"officeCode\" = '1'",  
  
  "intent": "filter employees by office location",  
  
  "tables": ["employees"],  
  
  "columns": ["*"],  
  
  "filters": ["officeCode = 1"],  
  
  "joins": [],  
  
  "aggregation": []  
}
```

Question 56: Orders with status Shipped

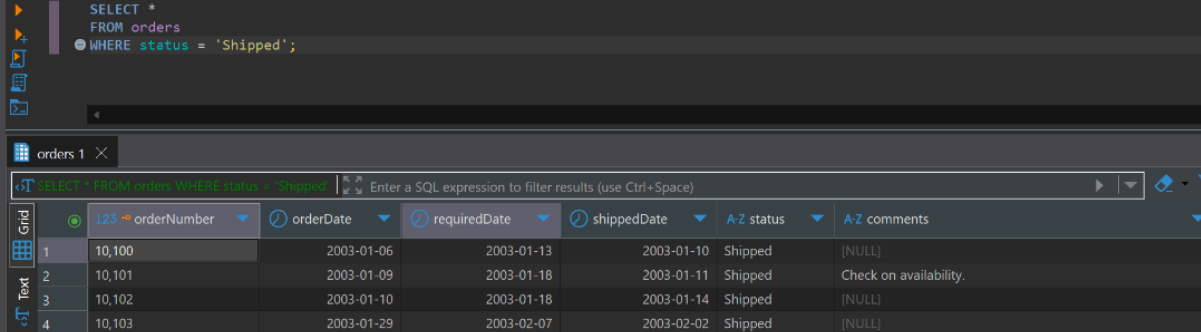
SQL Query

```
SELECT *  
FROM orders  
WHERE status = 'Shipped';
```

Explanation

This retrieves only shipped orders.

Result



The screenshot shows a database query interface. At the top, the SQL query is entered: `SELECT * FROM orders WHERE status = 'Shipped';`. Below the query, the results are displayed in a table. The table has columns: `orderNumber`, `orderDate`, `requiredDate`, `shippedDate`, `status`, and `comments`. The results show four rows of data, all with a status of 'Shipped'.

	orderNumber	orderDate	requiredDate	shippedDate	status	comments
1	10,100	2003-01-06	2003-01-13	2003-01-10	Shipped	[NULL]
2	10,101	2003-01-09	2003-01-18	2003-01-11	Shipped	Check on availability.
3	10,102	2003-01-10	2003-01-18	2003-01-14	Shipped	[NULL]
4	10,103	2003-01-29	2003-02-07	2003-02-02	Shipped	[NULL]

JSON

```
{  
  "question": "Orders with status Shipped",  
  "sql": "SELECT * FROM orders WHERE status = 'Shipped'",  
  "intent": "filter orders by status",  
  "tables": ["orders"],  
  "columns": ["*"],  
  "filters": ["status = Shipped"],  
  "joins": [],  
  "aggregation": []  
}
```


Question 57: Customers from France

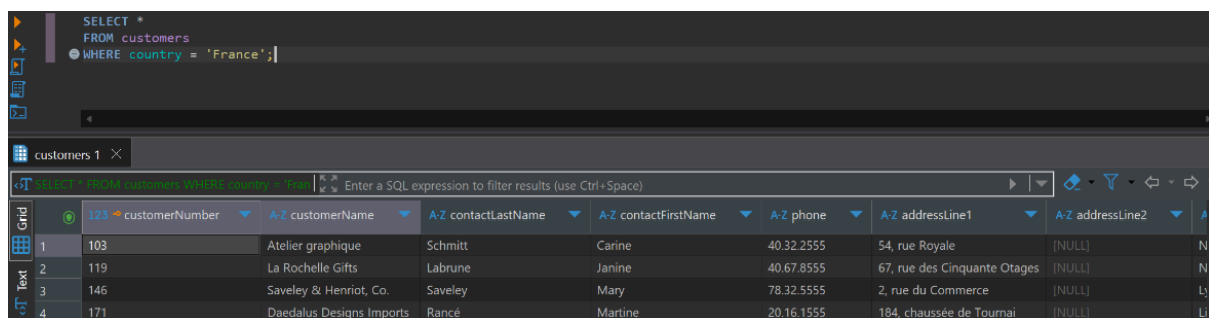
SQL Query

```
SELECT *  
FROM customers  
WHERE country = 'France';
```

Explanation

This filters customers located in France.

Result



The screenshot shows a database query interface. At the top, the SQL query is entered: `SELECT * FROM customers WHERE country = 'France';`. Below the query, the results are displayed in a table with 9 columns: `customerNumber`, `customerName`, `contactLastName`, `contactFirstName`, `phone`, `addressLine1`, `addressLine2`, and `country`. The results show 4 rows of data for customers located in France.

	customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1	addressLine2	country
1	103	Atelier graphique	Schmitt	Carine	40.32.2555	54, rue Royale	[NULL]	N
2	119	La Rochelle Gifts	Labruno	Janine	40.67.8555	67, rue des Cinquante Otages	[NULL]	N
3	146	Saveley & Henriot, Co.	Saveley	Mary	78.32.5555	2, rue du Commerce	[NULL]	N
4	171	Daedalus Designs Imports	Rance	Martine	20.16.1555	184, chaussée de Tournai	[NULL]	N

JSON

```
{  
  
  "question": "Customers from France",  
  
  "sql": "SELECT * FROM customers WHERE country = 'France'",  
  
  "intent": "filter customers by country",  
  
  "tables": ["customers"],  
  
  "columns": ["*"],  
  
  "filters": ["country = France"],  
  
  "joins": [],  
  
  "aggregation": []  
}
```

Question 58: Products from a specific vendor

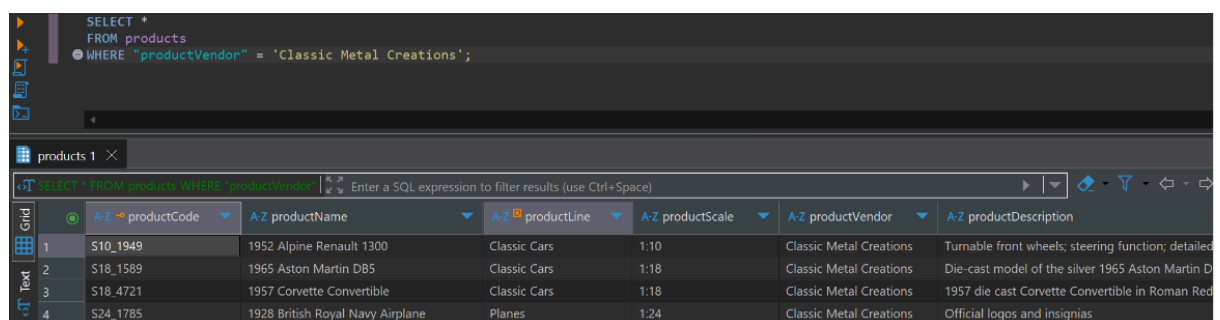
SQL Query

```
SELECT *  
FROM products  
WHERE "productVendor" = 'Classic Metal Creations';
```

Explanation

This retrieves products from a specific vendor.

Result



The screenshot shows a SQL query editor with the following query:

```
SELECT *  
FROM products  
WHERE "productVendor" = 'Classic Metal Creations';
```

Below the query editor, a table titled "products 1" displays the results. The table has 7 columns: productCode, productName, productLine, productScale, productVendor, and productDescription. The results are as follows:

	productCode	productName	productLine	productScale	productVendor	productDescription
1	S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; steering function; detailed
2	S18_1589	1965 Aston Martin DB5	Classic Cars	1:18	Classic Metal Creations	Die-cast model of the silver 1965 Aston Martin D
3	S18_4721	1957 Corvette Convertible	Classic Cars	1:18	Classic Metal Creations	1957 die cast Corvette Convertible in Roman Red
4	S24_1785	1928 British Royal Navy Airplane	Planes	1:24	Classic Metal Creations	Official logos and insignias

JSON

```
{  
  
  "question": "Products from a specific vendor",  
  
  "sql": "SELECT * FROM products WHERE \"productVendor\" = 'Classic Metal  
Creations'",  
  
  "intent": "filter products by vendor",  
  
  "tables": ["products"],  
  
  "columns": ["*"],  
  
  "filters": ["productVendor = Classic Metal Creations"],  
  
  "joins": [],  
  
  "aggregation": []  
}
```

Question 59: Payments between 2000 and 10000

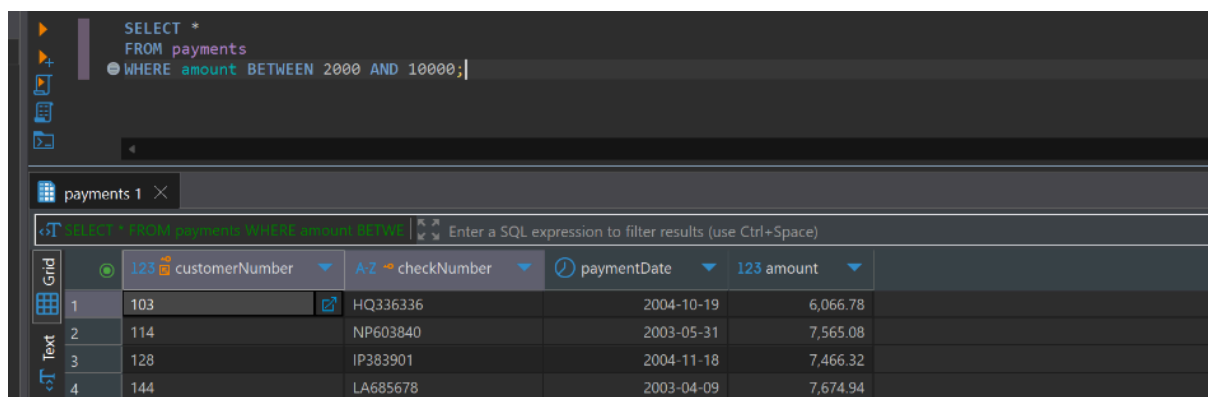
SQL Query

```
SELECT *  
FROM payments  
WHERE amount BETWEEN 2000 AND 10000;
```

Explanation

This filters payments within a specific numeric range.

Result



The screenshot shows a database query editor with the following SQL query:

```
SELECT *  
FROM payments  
WHERE amount BETWEEN 2000 AND 10000;
```

Below the query editor, a table titled "payments 1" displays the results. The table has five columns: "customerNumber", "checkNumber", "paymentDate", "amount", and an unlabeled column. The results are as follows:

	customerNumber	checkNumber	paymentDate	amount	
1	103	HQ336336	2004-10-19	6,066.78	
2	114	NP603840	2003-05-31	7,565.08	
3	128	IP383901	2004-11-18	7,466.32	
4	144	LA685678	2003-04-09	7,674.94	

JSON

```
{  
  "question": "Payments between 2000 and 10000",  
  "sql": "SELECT * FROM payments WHERE amount BETWEEN 2000 AND 10000",  
  "intent": "filter payments by range",  
  "tables": ["payments"],  
  "columns": ["*"],  
  "filters": ["amount BETWEEN 2000 AND 10000"],  
  "joins": [],  
  "aggregation": []  
}
```

Question 60: Customers whose name starts with A

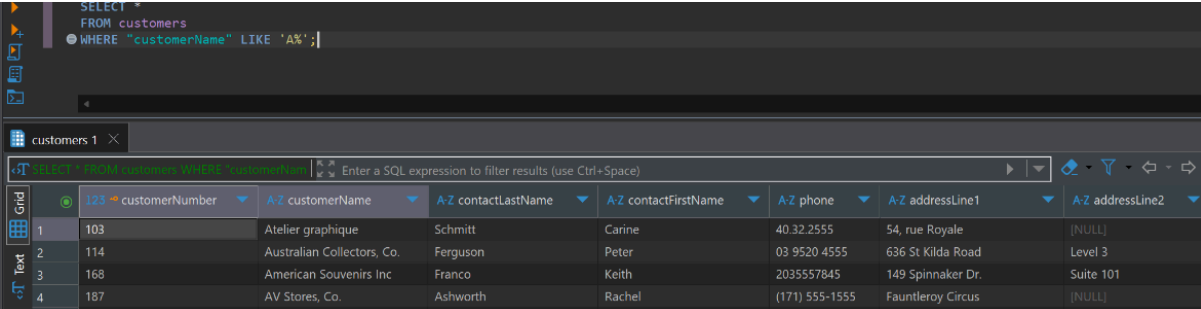
SQL Query

```
SELECT *  
FROM customers  
WHERE "customerName" LIKE 'A%';
```

Explanation

This filters customers whose names start with letter A.

Result



The screenshot shows a database query tool interface. The top panel displays the SQL query: `SELECT * FROM customers WHERE "customerName" LIKE 'A%';`. Below the query, a table titled "customers 1" shows the results. The table has 8 columns: customerNumber, customerName, contactLastName, contactFirstName, phone, addressLine1, and addressLine2. The results are filtered to show only customers whose names start with 'A'.

	customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1	addressLine2
1	103	Atelier graphique	Schmitt	Carine	40 32 2555	54, rue Royale	[NULL]
2	114	Australian Collectors, Co.	Ferguson	Peter	03 9520 4555	636 St Kilda Road	Level 3
3	168	American Souvenirs Inc	Franco	Keith	2035557845	149 Spinnaker Dr.	Suite 101
4	187	AV Stores, Co.	Ashworth	Rachel	(171) 555-1555	Fauntleroy Circus	[NULL]

JSON

```
{  
  
  "question": "Customers whose name starts with A",  
  
  "sql": "SELECT * FROM customers WHERE \"customerName\" LIKE 'A%'",  
  
  "intent": "filter customers by name pattern",  
  
  "tables": ["customers"],  
  
  "columns": ["*"],  
  
  "filters": ["customerName LIKE A%"],  
  
  "joins": [],  
  
  "aggregation": []  
}
```